

LAB REPORT

NAME-Divyanshu Singh

Sap-500122856

DevOpsAndCont

Contraization And Devops

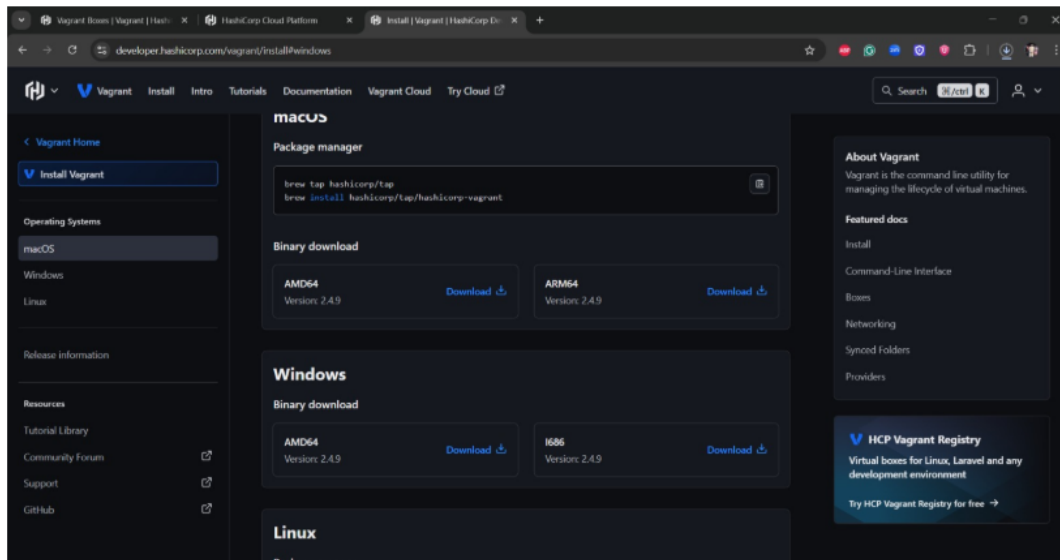
Lab 1: Vagrant & Nginx Setup

Student Details

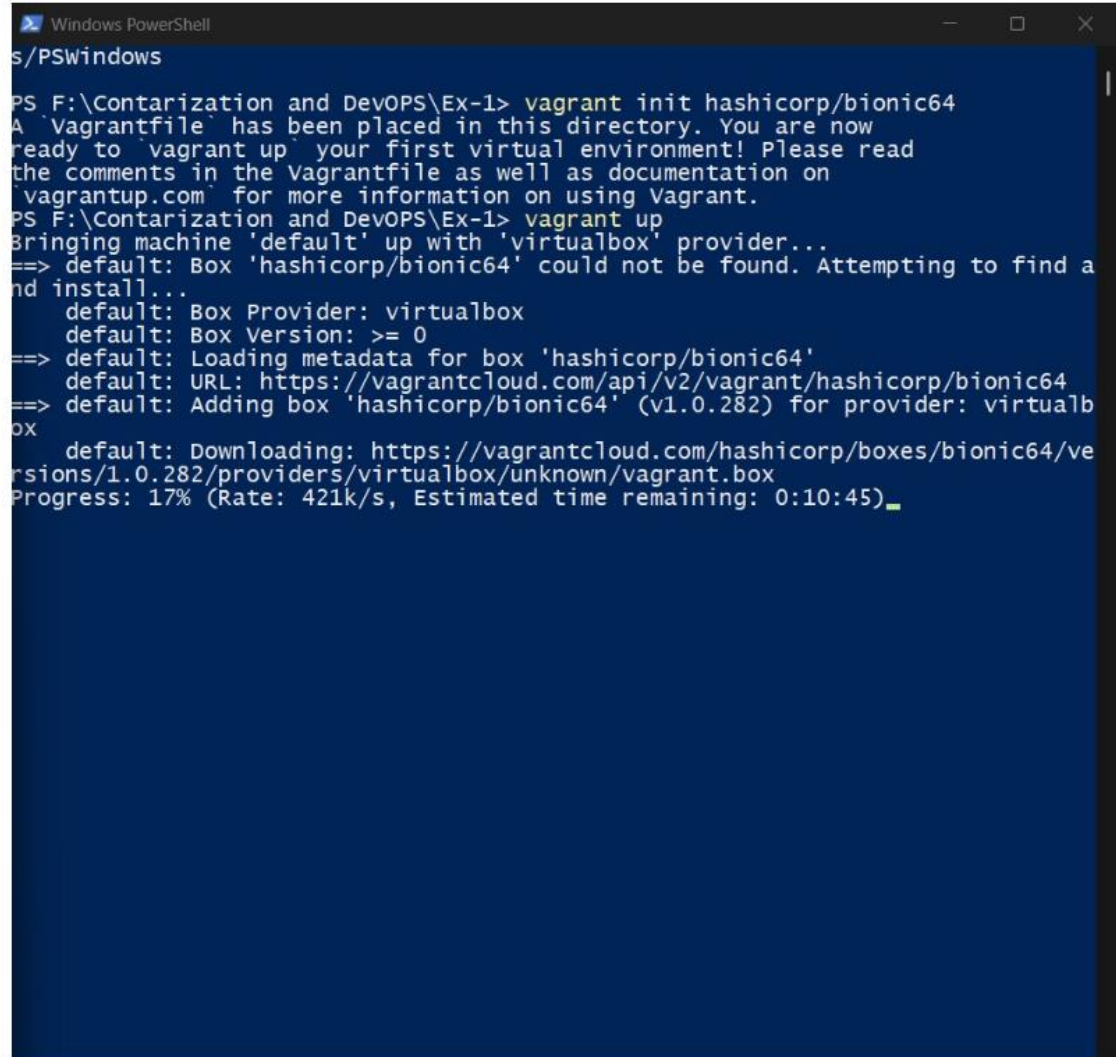
- Name: Divyanshu Singh
- Course: B.Tech
- SAP ID: 500122856
- Subject: DevOps Lab
- Semester: 6

Step-by-Step Procedure

Step 1: Installing Vagrant (AMD 64)



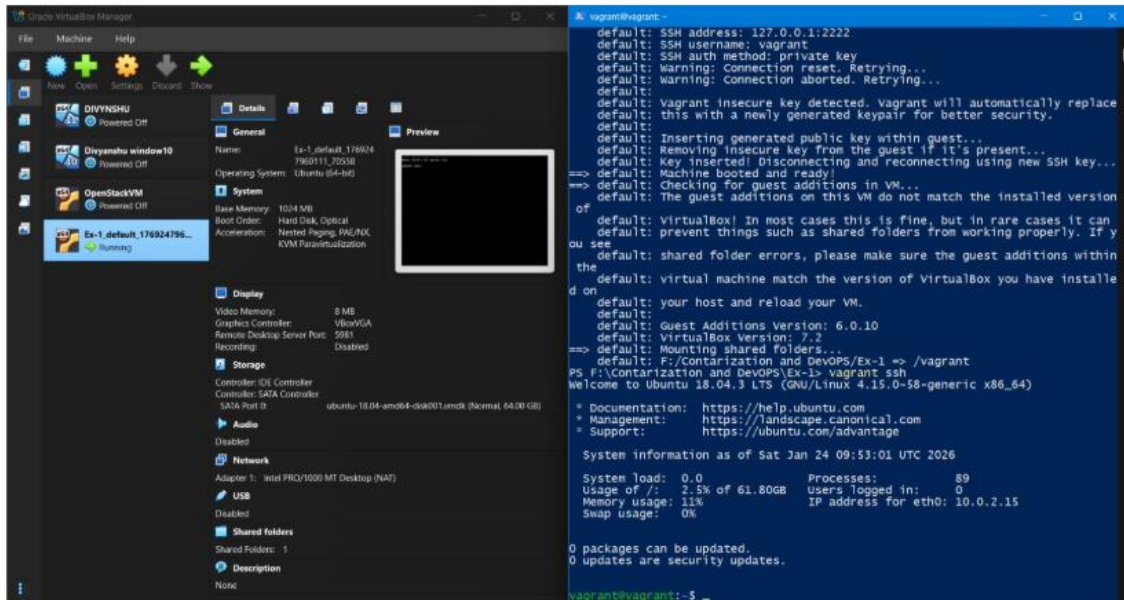
Step 2: After Vagrant Installation – Run Vagrant in Terminal



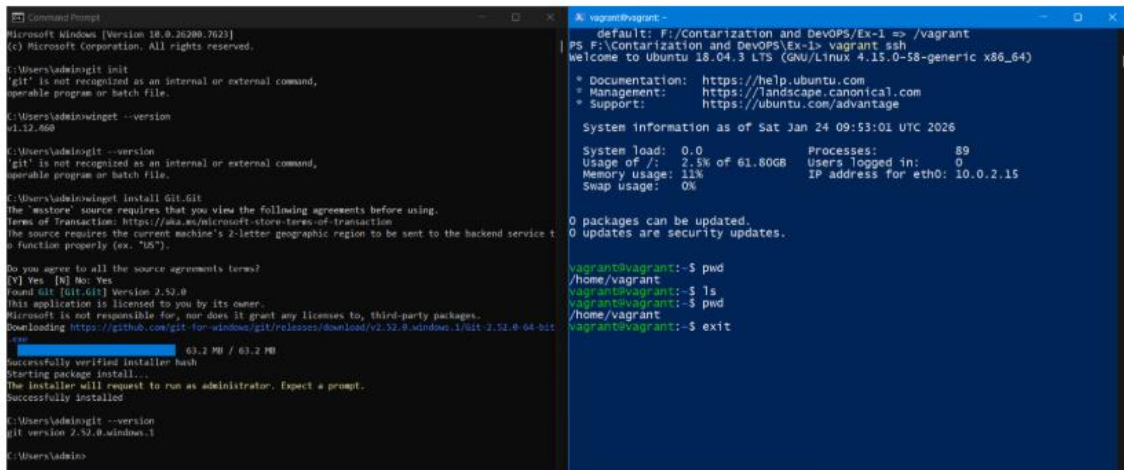
```
Windows PowerShell
s/PSWindows

PS F:\Contarization and DevOPS\Ex-1> vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
PS F:\Contarization and DevOPS\Ex-1> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find a
and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualb
ox
    default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/ve
rsions/1.0.282/providers/virtualbox/unknown/vagrant.box
Progress: 17% (Rate: 421k/s, Estimated time remaining: 0:10:45)
```

Step 3: Virtual Machine Created in VirtualBox



Step 4: Checking & Running Linux Commands



Step 5: Sudo Apt Update

```
System load: 0.43 Processes: 92
Usage of /: 2.5% of 61.80GB Users logged in: 0
Memory usage: 11% IP address for eth0: 10.0.2.15
Swap usage: 0%

0 packages can be updated.
0 updates are security updates.

Last login: Sat Jan 24 10:00:01 2026 from 10.0.2.2
vagrant@vagrant:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu bionic InRelease
Get:2 http://security.ubuntu.com/ubuntu bionic-security InRelease [102 kB]
Get:3 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [102 kB]
Get:4 http://security.ubuntu.com/ubuntu bionic-security/main i386 Packages [1,880 kB]
Get:5 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [102 kB]
Get:6 http://archive.ubuntu.com/ubuntu bionic-updates/main i386 Packages [1,665 kB]
Get:7 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [3,045 kB]
Get:8 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [2,717 kB]
Get:9 http://archive.ubuntu.com/ubuntu bionic-updates/main Translation-en [554 kB]
Get:10 http://archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [1,347 kB]
Get:11 http://security.ubuntu.com/ubuntu bionic-security/main Translation-en [468 kB]
Get:12 http://security.ubuntu.com/ubuntu bionic-security/restricted amd64 Packages [1,317 kB]
Get:13 http://archive.ubuntu.com/ubuntu bionic-updates/restricted i386 Packages [39.7 kB]
Get:14 http://archive.ubuntu.com/ubuntu bionic-updates/restricted Translation-en [187 kB]
Get:15 http://archive.ubuntu.com/ubuntu bionic-updates/universe amd64 Packages [1,915 kB]
Get:16 http://archive.ubuntu.com/ubuntu bionic-updates/universe i386 Packages [1,663 kB]
Get:17 http://security.ubuntu.com/ubuntu bionic-security/restricted i386 Packages [33.0 kB]
Get:18 http://security.ubuntu.com/ubuntu bionic-security/restricted Translation-en [182 kB]
Get:19 http://archive.ubuntu.com/ubuntu bionic-updates/universe Translation-en [421 kB]
```


Step 6: Install Nginx

```
2: packages can be upgraded: run: apt list --upgradable to see them
vagrant@vagrant:~$ sudo apt install -y nginx
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjpeg8
  libjpeg-turbo8 libpng16-0 libnginx-mod-http-geoip libnginx-mod-http-image-filter
  libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6 libx11-6
  libx11-data libxcb1 libxpm4 nginx-common nginx-core
Suggested packages:
  libgd-tools fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjpeg8
  libjpeg-turbo8 libpng16-0 libnginx-mod-http-geoip libnginx-mod-http-image-filter
  libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6 libx11-6
  libx11-data libxcb1 libxpm4 nginx nginx-common nginx-core
0 upgraded, 21 newly installed, 0 to remove and 294 not upgraded.
Need to get 3,195 kB of archives.
After this operation, 11.4 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libjpeg-turbo8
  amd64 1.5.2-0ubuntu5.18.04.6 [111 kB]
Get:2 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libxcb1 amd64
  1.13-2-ubuntu18.04 [45.5 kB]
Get:3 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libx11-data a
  ll 2:1.6.4-3ubuntu0.4 [114 kB]
7% [3 libx11-data 34.8 kB/114 kB 31%]
```

Step 7: Start Nginx Service

```
Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
Setting up libnginx-mod-http-xslt-filter (1.14.0-0ubuntu1.11) ...
Setting up libxcb1:amd64 (1.13-2-ubuntu18.04) ...
Setting up libnginx-mod-http-geoip (1.14.0-0ubuntu1.11) ...
Setting up libx11-data (2:1.6.4-3ubuntu0.4) ...
Setting up libwebp6:amd64 (0.6.1-2ubuntu0.18.04.2) ...
Setting up libjpeg8:amd64 (8c-2ubuntu8) ...
Setting up fontconfig-config (2.12.6-0ubuntu2) ...
Setting up libnginx-mod-stream (1.14.0-0ubuntu1.11) ...
Setting up libx11-6:amd64 (2:1.6.4-3ubuntu0.4) ...
Setting up libtiff5:amd64 (4.0.9-5ubuntu0.10) ...
Setting up libxpm4:amd64 (1:3.5.12-1ubuntu0.18.04.2) ...
Setting up libfontconfig1:amd64 (2.12.6-0ubuntu2) ...
Setting up libgd3:amd64 (2.2.5-4ubuntu0.5) ...
Setting up libnginx-mod-http-image-filter (1.14.0-0ubuntu1.11) ...
Setting up nginx-core (1.14.0-0ubuntu1.11) ...
Setting up nginx (1.14.0-0ubuntu1.11) ...
Processing triggers for ureadahead (0.100.0-21) ...
Processing triggers for ufw (0.36-0ubuntu0.18.04.1) ...
Processing triggers for libc-bin (2.27-3ubuntu1) ...
vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>
```

Step 8: CPU Performance Check



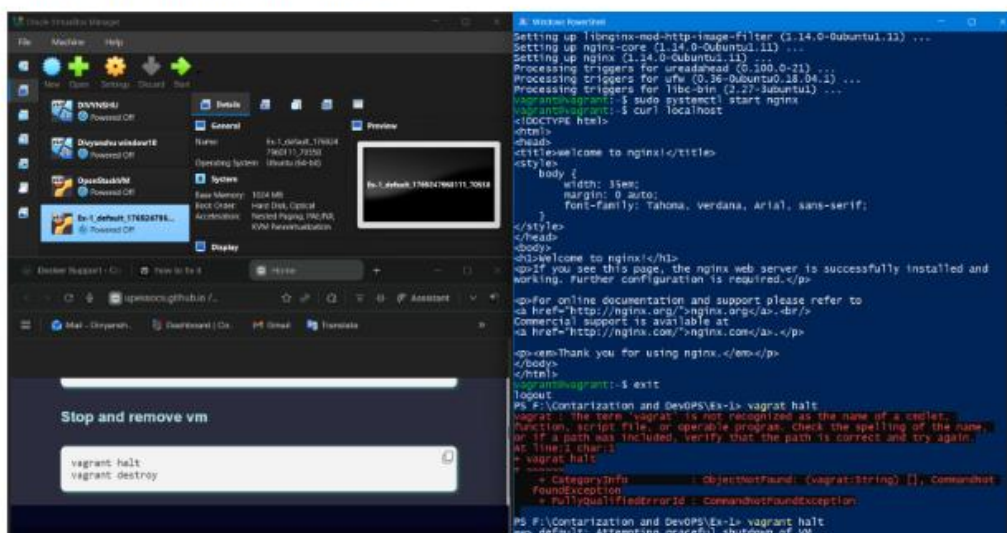
Step 9: Memory Performance Check



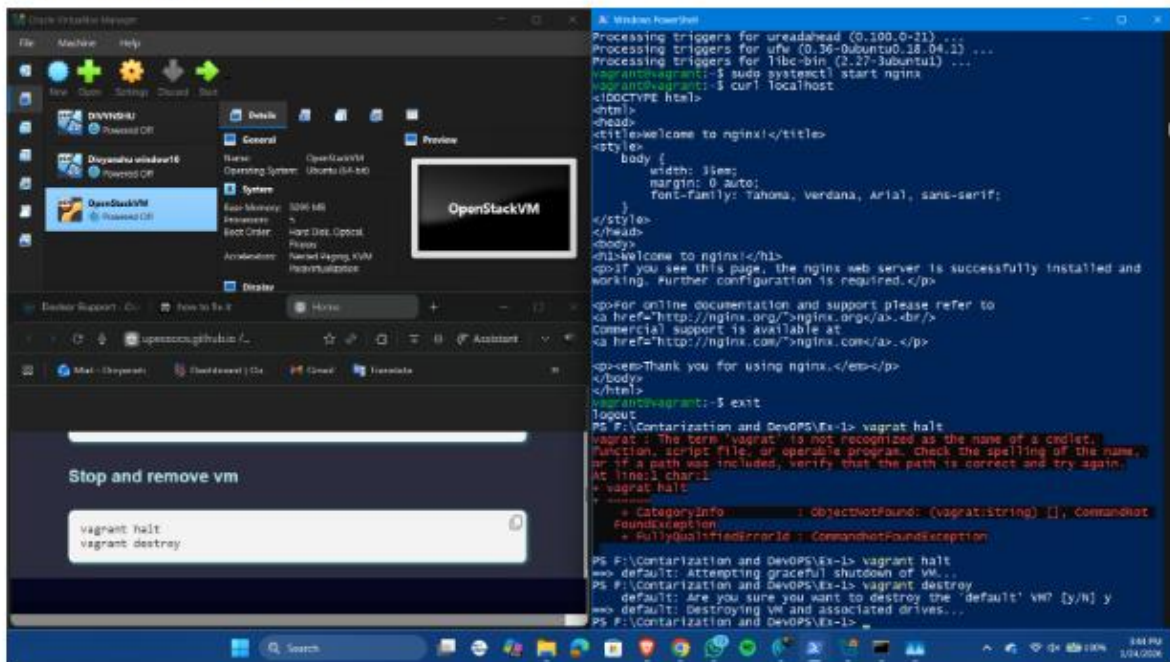
Step 10: Performance After Halt



Step 11: CPU Performance Check After Halt



Step 12: Destroy the Virtual Machine



Result

Vagrant was successfully installed and configured.

A virtual machine was created using VirtualBox, Nginx was installed and executed, system performance was monitored, and the VM was halted and destroyed successfully.

Contraization And Devops

Lab 2

Student Details

- Name: Divyanshu Singh
- Course: B.Tech
- SAP ID: 500122856
- Subject: DevOps Lab
- Semester: 6

Step-by-Step Procedure

- Step 1: Checking Docker Installation / Initial Setup

```
divyanshu@ubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ sudo docker --version
docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

Step 2: Installing Docker Packages

```
divyanshu@ubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ sudo docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
c3d29c2bd92508d43cf407aea0cefeae679602438eadcaa0d1866404ffcef40b
```

Step 3: Verifying Docker Version

```
divyanshu@ubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
c3d29c2bd925   nginx     "/docker-entrypoint..." 14 seconds ago Up 13 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
adoring_stonebraker
```

Step 3: Verifying Docker Version

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
c3d29c2bd925	nginx	"/docker-entrypoint..."	14 seconds ago	Up 13 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ S_
```

Step 4: Running Docker Service

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
c3d29c2bd925	nginx	"/docker-entrypoint..."	14 seconds ago	Up 13 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ sudo docker stop c3d
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c3d29c2bd925	nginx	"/docker-entrypoint..."	About a minute ago	Exited (0) 8 seconds ago		adori

Step 5: Running a Docker Container

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ sudo docker rm c3d
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
59c5c4e90bb	java_test3:01	"java hello"	30 hours ago	Exited (1) 30 hours ago		cool_hypatia
575db00d7db	java_test:01	"java hello"	30 hours ago	Exited (1) 30 hours ago		eloquent_bassi
ec52f3d7ed2c	java_test:01	"java hello"	30 hours ago	Exited (1) 30 hours ago		peaceful_blackburn
ff771f0e7c0	23f699328561	"java hello"	30 hours ago	Exited (1) 30 hours ago		madu_nash

Step 6: Listing Docker Images and Containers

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ docker rmi nginx
```

```
Untagged: nginx:latest
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
java_test3	01	5fa71e7137b8	30 hours ago	773MB
java_test	01	5fa71e7137b8	30 hours ago	773MB
<none>	<none>	23f699328561	30 hours ago	773MB
<none>	<none>	cbdbd831c7f7	3 days ago	773MB
myrepo/java-app	1.0	ce01aef35f93	4 days ago	773MB
ubuntu	latest	493218ed0f40	2 weeks ago	78.1MB
ubuntu	22.04	65c77cbc27c2	3 weeks ago	77.9MB
hello-world	latest	1b44b5a3e06a	5 months ago	10.1kB

Deploying NGINX Using Different Base Images and Comparing Image Layers

Lab Objectives

After completing this lab, students will be able to:

- Deploy NGINX using:
 - Official `nginx` image
 - Ubuntu-based image
 - Alpine-based image
 - Understand Docker image layers and size differences
 - Compare performance, security, and use-cases of each approach
 - Explain real-world use of NGINX in containerized systems
-

Prerequisites

- Docker installed and running
 - Basic knowledge of:
 - `docker run`
 - `Dockerfile`
 - Port mapping
 - Linux command basics
-

Part 1: Deploy NGINX Using Official Image (Recommended Approach)

Step 1: Pull the Image

```
docker pull nginx:latest
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-3$ docker pull nginx:latest
latest: Pulling from library/nginx
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

Step 2: Run the Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-3$ docker run -d --name nginx-official -p 8080:80 nginx
752d40d6c42997198006fb58d699be1c97e72bb94e70afa6372df6f5c9cc64aa
```

Step 3: Verify [🔗](#)

```
curl http://localhost:8080
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-3$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
```



```
divyanshu@buntugDESKTOP-GVMDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-3$ docker build -t nginx-ubuntu .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   681kB
Step 1/4 : FROM ubuntu:22.04
--> 65c77cbc27c2
Step 2/4 : RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lists/*
--> Running in 384ee667a5f1
Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1297 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [62.6 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [6388 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [2677 kB]
```

You should see the NGINX welcome page.

Part 2: Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

```
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

 CreateFiles

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

CreateFiles —

Observations

```
docker images nginx-ubuntu
```

CreateFiles

- Much larger image size
- More layers
- Full OS utilities available

Part 3: Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-alpine .
```

```
divyanshu@ubuntu@DESKTOP-GW4QHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-3$ docker build -t nginx-alpine -f Second.Dockerfile .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   681kB
Step 1/4 : FROM alpine:latest
--> a40c03cbb81c
Step 2/4 : RUN apk add --no-cache nginx
--> Running in 5f788c756d43
(1/2) Installing pcre2 (10.47-r0)
(2/2) Installing nginx (1.28.2-r0)
     Executing nginx-1.28.2-r0.pre-install
     Executing nginx-1.28.2-r0.post-install
Executing busybox-1.37.0-r30.trigger
OK: 9.9 MiB in 18 packages
--> Removed intermediate container 5f788c756d43
--> c0464eaf7552
Step 3/4 : EXPOSE 80
--> Running in 2fe05e899274
--> Removed intermediate container 2fe05e899274
--> 6ec9748fcd1
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
--> Running in dd22c9f1e09
```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-3$ docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
1f4fcd3050226ab4099deb43e617c08c366c7d73f05a6c55fb328f524847c1d
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-3$ docker images nginx-alpine
```

Observations

```
docker images nginx-alpine
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-3$ docker images nginx-alpine
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx-alpine latest 00f7ccab02f6 28 seconds ago 10.4MB
```

- Extremely small image
- Fewer packages
- Faster pull and startup time

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-3$ docker images | grep nginx
nginx-alpine latest 00f7ccab02f6 40 seconds ago 10.4MB
nginx-ubuntu latest 224c4f3e76b0 4 minutes ago 134MB
```

Typical result (approx):

Image Type	Size
nginx:latest	~140 MB
nginx-ubuntu	~220+ MB
nginx-alpine	~25–30 MB

Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```

Observations:

- Ubuntu has many filesystem layers
- Alpine has minimal layers
- Official NGINX image is optimized but heavier than Alpine

Part 5: Functional Tasks Using NGINX

Task 1: Serve Custom HTML Page

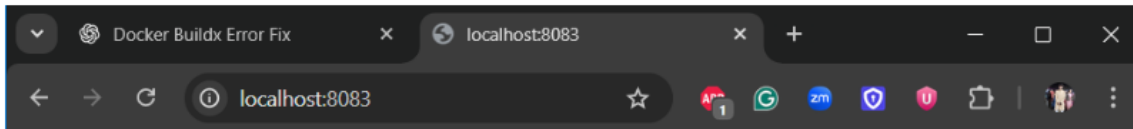
```
mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

```
divyanshubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-3$ mkdir html
divyanshubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-3$ echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

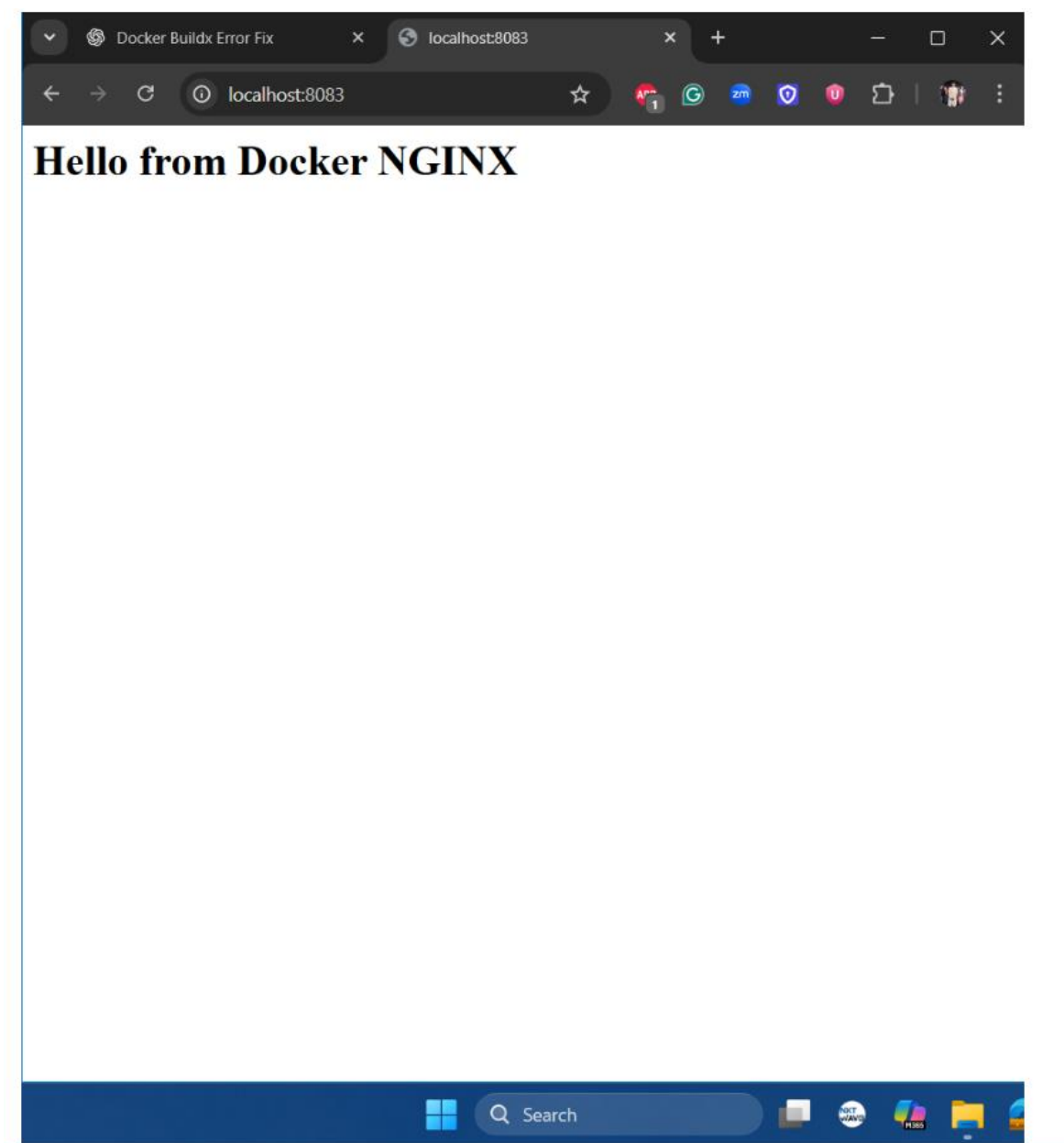
Run:

```
docker run -d \
-p 8083:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx
```

```
divyanshubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-3$ docker run -d -p 8083:80 -v $(pwd)/html:/usr/share/nginx/html nginx
a5b121504cf7bdfcabf4c9ed406746274662ba4fe90efee6e9862ca552bdc86c
```



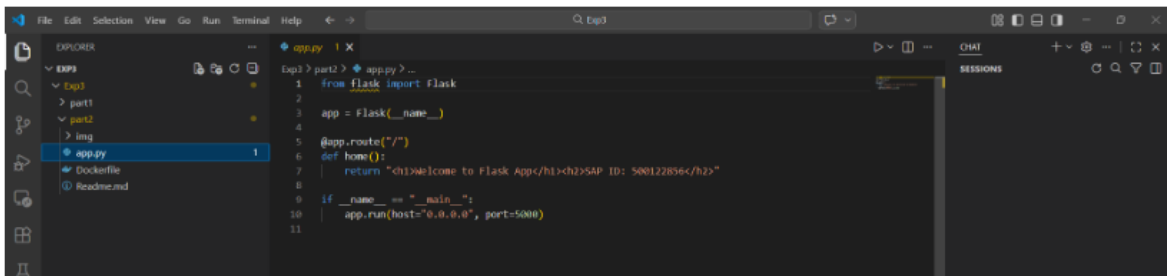
Hello from Docker NGINX



Hosting Flask app on docker

Create docker file

1. create separate folder
2. create a python program:



1. create docker file with commands:

```
FROM python:3.10-slim

WORKDIR /app

RUN pip install flask

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

```
Dockerfile > ...
1
2 FROM python:3.10-slim
3
4
5 WORKDIR /app
6
7 RUN pip install flask
8
9 COPY app.py .
10
11 EXPOSE 5000
12
13 CMD ["python", "app.py"]
14
```

Build and run image

1. Build the image from the docker file

```
docker build -t flask-sapid-app .
```



1. Check Image:

```
docker images
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin/Downloads/Exp3/Exp3/part2$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
flask-sapid-app     latest          5658d4bb7d68   7 seconds ago  134MB
nginx-alpine        latest          00f7ccab02f6   4 days ago     10.4MB
nginx-ubuntu        latest          224c4f3e76b0   4 days ago     134MB
class_ex-2          1.0            bb45267f6631   5 days ago     1.21GB
nginx                latest          5cdef4ac3335   7 days ago     161MB
```

1. Run the image:

```
docker run -d -p 8080:5000 flask-sapid-app:3.0
```

1. Run the image:

```
docker run -d -p 8080:5000 flask-sapid-app:3.0
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/c/Users/admin/Downloads/Exp3/Exp3/part2$ docker run -d -p 8080:500 flask-sapid-ap  
p  
5651132db85b45fac00e76a241efb90bc37cb9ea43e217e211a1a0a6094f3592
```

Result

Python program running



Thank You

DevOpsAndCont

Experiment 4: Docker Essentials

Dockerfile .dockerignore tagging publishing

Containerizing Applications with Dockerfile

Step-1: Create a Directory for Python application

```
mkdir my-flask-app
cd my-flask-app
```

```
PS F:\Containerization and DevOPS\Containerization And Devops_Lab\Lab-4> wsl
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-4$ cd my-flask-app
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-2:- Create an **app.py** :******

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step-3:- Create **requirements.txt** :******

```
Flask==2.3.3
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ cat requirements.txt
Flask==2.3.3
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-4: Create ****Dockerfile****

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ cat Dockerfile
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-5:- Create a **.dockerignore :

```
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
```



```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-flask-app$ cat .dockerignore
# Python files
__pycache__/*
*.pyc
*.pyo
*.pyd

# Environment files
.env
.env.*
env/*
venv/*

# IDE files
.vscode/*
.idea/*

# Git files
.git/*
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/*

# Test files
tests/*
test_*.py
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-flask-app$
```

Why `.dockerignore` is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Step-6: Build image from Dockerfile

```
docker build -t my-flask-app .
```

```
divyanshubuntu@DESKTOP-GWQXQP1:/mnt/1/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker build -t my-flask-app .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
3.9-slim: Pulling from library/python
38513bd72563: Pulling fs layer
b3ec39b36ae8: Pulling fs layer
fc7443084902: Pulling fs layer
ea56f685404a: Pulling fs layer
ea56f685404a: Waiting
b3ec39b36ae8: Download complete
ea56f685404a: Verifying Checksum
ea56f685404a: Download complete
fc7443084902: Download complete
38513bd72563: Download complete
38513bd72563: Pull complete
b3ec39b36ae8: Pull complete
fc7443084902: Pull complete
ea56f685404a: Pull complete
Digest: sha256:2097f6918b16bd338d3860f261f53f144965f75599aabi1acda1e13cf1731b1b
Status: Downloaded newer image for python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Running in 99ac1f50fc2f
--> Removed intermediate container 99ac1f50fc2f
--> 13688294599e
Step 3/7 : COPY requirements.txt .
--> 0edcab2f5c64
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Running in 6a6b4c4314c4
Collecting Flask==2.3.3
  Downloading flask-2.3.3-py3-none-any.whl (96 kB)
```

Step-7:- Verify Image

docker images

```
divyanshubuntu@DESKTOP-GWQXQP1:/mnt/1/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-flask-app	latest	5a96110b8f77	36 seconds ago	132MB
wordpress	<none>	901232ac681e	8 days ago	752MB
assignment_1-backend	latest	ee2552e95f3a	4 weeks ago	138MB
assignment_1-database	latest	9cf734661ac1	4 weeks ago	274MB
jenkins/jenkins	lts	3e88dcb3d033	6 weeks ago	487MB
advancedtask-fastapi-app	latest	20e4b41820e5	8 weeks ago	140MB
python	3.9-slim	085da638e1b8	6 months ago	122MB
node	18-alpine	ee77c6cd7c18	13 months ago	127MB
ghcr.io/k3d-io/k3d-proxy	5.8.3	49c793b9faf6	14 months ago	63.1MB
rancher/k3s	v1.31.5-k3s1	efee5d76faac	15 months ago	222MB
mysql	<none>	5107333e08a8	2 years ago	501MB

Step-8: Add Tag in Image

```
docker build -t my-flask-app:1.0 .
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker build -t my-flask-app:1.0 .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 13688294599e
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 0edcab2f5c64
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> f386e722f726
Step 5/7 : COPY app.py .
--> Using cache
--> 282de7bd9e2c
Step 6/7 : EXPOSE 5000
--> Using cache
--> 302111148efd
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> 5a96110b8f27
Successfully built 5a96110b8f27
Successfully tagged my-flask-app:1.0
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-9:- Add multiple tags

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker build -t my-flask-app:latest -t my-flask-app:1.0 .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 13688294599e
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 0edcab2f5c64
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> f386e722f726
Step 5/7 : COPY app.py .
--> Using cache
--> 282de7bd9e2c
Step 6/7 : EXPOSE 5000
--> Using cache
--> 302111148efd
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> 5a96110b8f27
Successfully built 5a96110b8f27
Successfully tagged my-flask-app:latest
Successfully tagged my-flask-app:1.0
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-10:- Add Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/ff/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker build -t username/my-flask-app:1.0 .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 13688294599e
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 0edcab2f5c64
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> f386e722f726
Step 5/7 : COPY app.py .
--> Using cache
--> 282de7bd9e2c
Step 6/7 : EXPOSE 5000
--> Using cache
--> 302111148efd
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> 5a96110b8f27
Successfully built 5a96110b8f27
Successfully tagged username/my-flask-app:1.0
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/ff/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-11:- Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

Step-12: View Image Details

docker images

```
divyanshubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
username/my-flask-app	1.0	5a96110b8f27	3 minutes ago	132MB
my-flask-app	1.0	5a96110b8f27	3 minutes ago	132MB
my-flask-app	latest	5a96110b8f27	3 minutes ago	132MB
my-flask-app	v1.0	5a96110b8f27	3 minutes ago	132MB
wordpress	<none>	901232ac681e	8 days ago	752MB
assignment_1-backend	latest	ee2553e95f3a	4 weeks ago	138MB
assignment_1-database	latest	9cf734661ac1	4 weeks ago	274MB
jenkins/jenkins	lts	3e88dcb3d033	6 weeks ago	487MB
advancedtask-fastapi-app	latest	28eb141820e5	8 weeks ago	140MB
python	3.9-slim	085da638e1b8	6 months ago	122MB
node	18-alpine	ee77c6cd7c18	13 months ago	127MB
ghcr.io/k3d-io/k3d-proxy	5.8.3	49c793b9faf6	14 months ago	63.1MB
rancher/k3s	v1.31.5-k3s1	efe65d76faac	15 months ago	222MB
mysql	<none>	5107333e08a8	2 years ago	501MB

Step-13:- Show image history

docker history my-flask-app

```
divyanshubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker history my-flask-app
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
5a96110b8f27	4 minutes ago	/bin/sh -c #(nop) CMD ["python" "app.py"]	0B	
302111148efd	4 minutes ago	/bin/sh -c #(nop) EXPOSE 5000	0B	
282de7b99e2c	4 minutes ago	/bin/sh -c #(nop) COPY file:444a3904a5c3d221...	228B	
f386c722f726	4 minutes ago	/bin/sh -c pip install --no-cache-dir -r req...	10.5MB	
0edcah2f5c64	4 minutes ago	/bin/sh -c #(nop) COPY file:f095d6d544b2123...	13B	
1368824599e	4 minutes ago	/bin/sh -c #(nop) WORKDIR /app	0B	
085da638e1b8	6 months ago	CMD ["python3"]	0B	buildkit.dockerfile.v0
<missing>	6 months ago	RUN /bin/sh -c set -eux; for src in idle3 p...	36B	buildkit.dockerfile.v0
<missing>	6 months ago	RUN /bin/sh -c set -eux; savedAptMark="\$(a...	39.5MB	buildkit.dockerfile.v0
<missing>	6 months ago	ENV PYTHON_SHA256=00e07d7c0f2f0cc00243d1ee8...	0B	buildkit.dockerfile.v0
<missing>	6 months ago	ENV PYTHON_VERSION=3.9.25	0B	buildkit.dockerfile.v0
<missing>	6 months ago	ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3...	0B	buildkit.dockerfile.v0
<missing>	6 months ago	RUN /bin/sh -c set -eux; apt-get update; a...	3.81MB	buildkit.dockerfile.v0
<missing>	6 months ago	ENV LANG=C.UTF-8	0B	buildkit.dockerfile.v0
<missing>	6 months ago	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...	0B	buildkit.dockerfile.v0
<missing>	6 months ago	# debian.sh --arch 'amd64' out/ 'trixie' '@L...	78.6MB	debuerreotype 0.16

Step-14:- Inspect image details

```
docker inspect my-flask-app
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker inspect my-flask-app
[
  {
    "Id": "sha256:5a96110b8f27861597fbc832b3f64ce3884a12525675079519777929ddcce93",
    "RepoTags": [
      "my-flask-app:1.0",
      "my-flask-app:latest",
      "my-flask-app:v1.0",
      "username/my-flask-app:1.0"
    ],
    "RepoDigests": [],
    "Parent": "sha256:302111148efdea13bd85b817aad4031abc6165376eb4f8f18db5df63ffe24ac",
    "Comment": "",
    "Created": "2026-04-30T13:33:46.287369536Z",
    "DockerVersion": "28.2.2",
    "Author": "",
    "Config": {
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048825C084DEBE9826995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=0ee07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "Cmd": [
        "python",
        "app.py"
      ],
      "WorkingDir": "/app"
    },
    "Architecture": "amd64",
    "Os": "linux",
    "Size": 132491037,
    "GraphDriver": {
      "Data": {
        "LowerDir": "/var/lib/docker/overlay2/fc94391240c41254fd609ba0bbcc19a20ef396ef659d5f1eb199fd2dff82ea4a/diff:/var/lib/docker/overlay2/71079681a275a7f0294185c2f1e0d7347af099a60220a57aa751e0a1d4ea03f6/diff:/var/lib/docker/overlay2/1626c5c067ffe34685b2db75ecba09cd181b7884ab0c4c481b6a530737ae08/diff:/var/lib/docker/overlay2/d192e0075ed4f8f69e2ee108c897d1a96182f2eb25cb30daec15f0e0565b85b/diff:/var/lib/docker/overlay2/2548be88f3978acbecb29f6680b483c0c/ed4f2eef0f/bcf286521db1ef49/c5"
      }
    }
  }
]
```

Step-15: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker run -d -p 5000:5000 --name flask-container my-flask-app
5c7569c9067b5770546084c356d42a3208a6970813816038e72de0e6c040ae0
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-16:- Test the application

```
curl http://localhost:5000
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ curl http://localhost:5000
Hello from Docker!divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-17:- View running containers

```
docker ps
```

```
Hello from Docker!divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And DEvops_Lab/Lab/Lab-4/my-flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
5c7569c9067b   my-flask-app   "python app.py"         About a minute Up About a minute   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
flask-container b1a77bec2779   jenkins/jenkins:lts     "/usr/bin/tini -- /u..." 2 weeks ago   Up 10 minutes
jenkins        f6e3d9adc07a   assignment_1-backend    "docker-entrypoint.s..." 4 weeks ago   Up 10 minutes
node_backend   785a12392bf4   assignment_1-database    "docker-entrypoint.s..." 4 weeks ago   Up 10 minutes (healthy)
postgres_db    1ccc305816d9   ghcr.io/k3d-io/k3d-proxy:5.8.3 "/bin/sh -c nginx-pr..." 6 weeks ago   Up 10 minutes   80/tcp, 0.0.0.0:41983->6443/tcp
k3d-mycluster-server1b 61c9080826f2   rancher/k3s:v1.31.5-k3s1 "/bin/k3d-entrypoint..." 6 weeks ago   Up 10 minutes
k3d-mycluster-server-0 6a8bf745da79   advancedtask-fastapi-app "uvicorn main:app --..." 8 weeks ago   Up 10 minutes   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
fastapi-app
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And DEvops_Lab/Lab/Lab-4/my-flask-app$
```

Step-18:- View container logs

```
docker logs flask-container
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And DEvops_Lab/Lab/Lab-4/my-flask-app$ docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [30/Apr/2026 13:40:22] "GET / HTTP/1.1" 200 -
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And DEvops_Lab/Lab/Lab-4/my-flask-app$
```

Step-19:- Stop container

```
docker stop flask-container
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And DEvops_Lab/Lab/Lab-4/my-flask-app$ docker stop flask-container
flask-container
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And DEvops_Lab/Lab/Lab-4/my-flask-app$
```

Step-20:- Start stopped container

```
docker start flask-container
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And DEvops_Lab/Lab/Lab-4/my-flask-app$ docker start flask-container
flask-container
```


Step-21:- Remove container

```
docker rm flask-container
```

```
divyanshubuntu@DESKTOP-GWHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker rm flask-container
Error response from daemon: cannot remove container "flask-container": container is running; stop the container before removing or force remove
divyanshubuntu@DESKTOP-GWHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Step-22:- Remove container forcefully

Since Container is in Running State, so we will remove it forcefully...

```
docker rm -f flask-container
```

```
divyanshubuntu@DESKTOP-GWHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker rm -f flask-container
flask-container
divyanshubuntu@DESKTOP-GWHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$
```

Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step-23: Create a Simple **Dockerfile.multistage** :

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```



```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4$ cat Dockerfile.multistage
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4$
```

Step-24:- Build regular image

```
docker build -t flask-regular .
```

```

divyanshubuntu@DESKTOP-GVHQZHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker build -t flask-regular .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with Buildkit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 13688294599e
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 0edcab2f5c64
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> f386e722f726
Step 5/7 : COPY app.py .
--> Using cache
--> 282de7bd9e2c
Step 6/7 : EXPOSE 5000
--> Using cache
--> 302111148efd
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> 5a96110b8f27
Successfully built 5a96110b8f27
Successfully tagged flask-regular:latest
divyanshubuntu@DESKTOP-GVHQZHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$

```

Step-25:- Build multi-stage image

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

```

divyanshubuntu@DESKTOP-GVHQZHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-4/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with Buildkit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/15 : FROM python:3.9-slim AS builder
--> 085da638e1b8
Step 2/15 : WORKDIR /app
--> Using cache
--> 13688294599e
Step 3/15 : COPY requirements.txt .
--> Using cache
--> 0edcab2f5c64
Step 4/15 : RUN python -m venv /opt/venv
--> Running in d53eb44ae35a
--> Removed intermediate container d53eb44ae35a
--> 12a9f96e9f5b
Step 5/15 : ENV PATH="/opt/venv/bin:$PATH"
--> Running in a911ff1824cc
--> Removed intermediate container a911ff1824cc
--> 6970d32a182d
Step 6/15 : RUN pip install --no-cache-dir -r requirements.txt
--> Running in bd387007a08a
Collecting Flask==2.3.3
  Downloading flask-2.3.3-py3-none-any.whl (96 kB)
    _____ 96.1/96.1 kB 5.4 MB/s eta 0:00:00
Collecting blinker>=1.6.2
  Downloading blinker-1.6.2-py3-none-any.whl (17 kB)
    _____ 17.0/17.0 kB 5.4 MB/s eta 0:00:00

```

Step-26:- Compare sizes

```
docker images | grep flask-
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker images | grep flask-
flask-multistage      latest      84b6ae88cea4 22 seconds ago 145MB
flask-regular         latest      5a96110b8f27 33 minutes ago 132MB
my-flask-app          1.0        5a96110b8f27 33 minutes ago 132MB
my-flask-app          latest      5a96110b8f27 33 minutes ago 132MB
my-flask-app          v1.0       5a96110b8f27 33 minutes ago 132MB
username/my-flask-app 1.0        5a96110b8f27 33 minutes ago 132MB
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$
```

Publishing to Docker Hub

Step-27:- Login to Docker Hub

```
docker login
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker login
Authenticating with existing credentials... [Username: divyanshusingh151103]
```

```
Info -> To login with a different account, run 'docker logout' followed by 'docker login'
```

```
Login Succeeded
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$
```

Step-28:- Tag image for Docker Hub

```
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker tag my-flask-app:latest divyanshusingh151103/my-flask-app:1.0
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker tag my-flask-app:latest divyanshusingh151103/my-flask-app:latest
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$
```

Step-29:- Push to Docker Hub

```
docker push username/my-flask-app:1.0
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker push divyanshusingh151103/my-flask-app:1.0
The push refers to repository [docker.io/divyanshusingh151103/my-flask-app]
22ba2a00d5a: Pushed
55168aba44f7: Pushed
5f9a4486b7f7: Pushed
535ee9e412e: Pushed
c8f6b54339a8: Mounted from library/python
298992e09a03: Mounted from library/python
4f237755fbae: Mounted from library/python
d7c97cb6f1fe: Mounted from library/python
```

Step-30:- Push Latest Image

```
docker push username/my-flask-app:latest
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker push divyanshusingh151103/my-flask-app:latest
The push refers to repository [docker.io/divyanshusingh151103/my-flask-app]
22ba2a008d5a: Layer already exists
55168aba44f7: Layer already exists
5f9a4486b7f7: Layer already exists
535eea9e412e: Layer already exists
c8f6b54339a8: Layer already exists
298992e09a03: Layer already exists
4f237755fbae: Layer already exists
d7c97cbef1fe: Layer already exists
```

Step-31:- Pull from Docker Hub (on another machine)

```
docker pull username/my-flask-app:latest
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker pull divyanshusingh151103/my-flask-app:latest
latest: Pulling from divyanshusingh151103/my-flask-app
Digest: sha256:380c770d343579e19c21517aa8c1401f8c63e30a2cc7f6d7d939727667a29d5d
Status: Image is up to date for divyanshusingh151103/my-flask-app:latest
docker.io/divyanshusingh151103/my-flask-app:latest
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-flask-app$
```

Step-32:- Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-flask-app$ docker run -d -p 5000:5000 divyanshusingh151103/my-flask-app:latest
7e60c8114de2d3923359325dd3062a66c2e927a1904b5a8ff53b43d9a3d3bef1
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-flask-app$
```

Node.js Application Example (Quick Version)**

Step-33:- Create a directory Node.js Application

```
mkdir my-node-app
cd my-node-app
```

```
PS F:\Containerization and DevOps\Containerization And Devops_Lab\Lab\Lab-4> wsl
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4$ cd my-node-app
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-4/my-node-app$
```

Step-34:- Create a **app.js**.

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-node-app$ cat app.js
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-node-app$
```

Step-35:- Create a **package.json** :

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-4/my-node-app$ cat package.json
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-4/my-node-app$
```

Step-36:- Create a Node.js Dockerfile

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-4/my-node-app$ cat Dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-4/my-node-app$
```

Step-37:- Build image

```
docker build -t my-node-app .
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-node-app$ docker build -t my-node-app .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM node:18-alpine
--> ee77c6cd7c18
Step 2/7 : WORKDIR /app
--> Running in e213987dc719
--> Removed intermediate container e213987dc719
--> 41b1dd4e845c
Step 3/7 : COPY package*.json ./
--> 3b08308aca97
Step 4/7 : RUN npm install --only-production
--> Running in fef66ce93b1d
npm warn config only use "--omit-dev" to omit dev dependencies from the install.
```

Step-38:- Run container

```
docker run -d -p 3000:3000 --name node-container my-node-app
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-node-app$ docker run -d -p 3000:3000 --name node-container my-node-app
11447e61f35658d721df15935635e5dfb2c99415cd586f95652c28692c4b6161
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-node-app$
```

Step-39:- Test on Terminal

```
curl http://localhost:3000
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-node-app$ curl http://localhost:3000
Hello from Node.js Docker!divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-4/my-node-app$
```

Step-40:- Test on Browser

The screenshot shows a web browser window with the address bar set to 'localhost:3000'. The page content displays 'Hello from Node.js Docker!'. The browser's developer tools are open at the bottom, showing the network tab with a single request to 'localhost:3000'.

Essential Docker Commands Cheatsheet

Command	Purpose	Example
<code>docker build</code>	Build image	<code>docker build -t myapp .</code>
<code>docker run</code>	Run container	<code>docker run -p 3000:3000 myapp</code>
<code>docker ps</code>	List containers	<code>docker ps -a</code>
<code>docker images</code>	List images	<code>docker images</code>
<code>docker tag</code>	Tag image	<code>docker tag myapp:latest myapp:v1</code>
<code>docker login</code>	Login to Dockerhub using username and password or token	<code>echo "token" docker login -u username --password-stdin</code>
<code>docker push</code>	Push to registry	<code>docker push username/myapp</code>
<code>docker pull</code>	Pull from registry	<code>docker pull username/myapp</code>
<code>docker rm</code>	Remove container	<code>docker rm container-name</code>
<code>docker rmi</code>	Remove image	<code>docker rmi image-name</code>
<code>docker logs</code>	View logs	<code>docker logs container-name</code>
<code>docker exec</code>	Execute command	<code>docker exec -it container-name bash</code>

Production Workflow:

```
# 1. Pull from registry
docker pull myapp:v1.0

# 2. Run in production
docker run -d -p 80:8080 --name prod-app myapp:v1.0

# 3. Monitor
docker logs -f prod-app
```

Key Takeaways

- 1. **Dockerfile** defines how to build your image
- 2. **.dockerignore** excludes unnecessary files (important for security and performance)
- 3. **Tagging** helps version and organize images
- 4. **Multi-stage builds** create smaller, more secure production images
- 5. **Docker Hub** allows sharing and distributing images
- 6. **Always test** images locally before publishing

DevOpsAndCont

opwf# Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
# Create a container that writes data
docker run -it --name test-container ubuntu /bin/bash

# Inside container:
echo "Hello World" > message.txt
cat message.txt # Shows "Hello World"
exit

# delete and make a new container
docker stop test-container
docker rm test-container
docker run -it --name test-container ubuntu /bin/bash
cat message.txt
# ERROR: File doesn't exist! Data was Lost.
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/ff/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$ docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
b40150c1c271: Pull complete
Digest: sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
Status: Downloaded newer image for ubuntu:latest
root@d9a70bfed87b:/#
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/ff/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$ docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
b40150c1c271: Pull complete
Digest: sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
Status: Downloaded newer image for ubuntu:latest
root@d9a70bfed87b:/# cat message.txt
cat: message.txt: No such file or directory
root@d9a70bfed87b:/#
```

Solution: Docker Volumes

Lab 2: Volume Types

1. Anonymous Volumes

```
# Create anonymous volume (auto-generated name)
docker run -d -v /app/data --name web1 nginx

# Check volume
docker volume ls
# Shows: anonymous volume with random hash

# Inspect container to see volume mount
docker inspect web1 | grep -A 5 Mounts
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-5$ docker run -d -v /app/data --name web1 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
3531af2bc2a9: Already exists
ce776bbcdad0: Pull complete
85c66128325a: Pull complete
4677c2a9a3d4: Pull complete
ff048f1f2159: Pull complete
677c63196868: Pull complete
801a1ad15b4e: Pull complete
Digest: sha256:6e23479198b998e5e25921dfff8455837c7636a67111a04a635cf1bb363d199dc
Status: Downloaded newer image for nginx:latest
e9ae9687bc10a31e143efb84b1e64ad68ba1bb5397463f7700f20b11a1328e0e
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-5$
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-5$ docker volume ls
DRIVER      VOLUME NAME
local       1c1b3d5ea8bd1d7a026c05ff1b50dc28d8a2b99473992631d231dbc545e6d414
local       2b7560209d7dd52c4ede810b0c0c22abf8db1e1b650c7ed6ef1a8a998b6b3d33
local       5d1106e39e6625503ea341fb677c22643ec3e1b777b4d0d8dd68718a3bbaf94ea
local       7b771ce4e592935b9d1e5ef6f3336ef669148c94777d921fd589bdacff31ebee
local       9e21c77f884ef3d1c552214f8687da8d252fd0699e9a11c7a245382f7436028f
local       087c3a07dc435d48bdb89129e6a146d77f80c429ef024870624c779f93ff7dff
local       457d9ec51623465f9f1dabeb412b4e01cd7d14ffc810eae776ea0c7bc99a06e5
local       551c22f645cc7324a47830fa173baaa0dc272c433a83a9ced999321ecd645848
local       5712fe64ba49e3596ed5e5cabc6d3b7dd890be0cca49aec7caeff313c9429a85
local       aee9db2b4cab8655472fc32d93dada40d38f324d8e0b1ee08233b47ad60b5528
local       app-volume
local       assignment_1_pgdata
local       c002d4a5e4a7bd3a87c8a2cb6aa33aa29ffff009361bdfda6f57d4ffh21ebe0a4
local       c76d238275c9d4701deeda5b032520fb1269a094b8ad2386fec330ea2fa989e9
local       d6e00f7334f07acc23e92ab4b6f6425844511735c6c24d75234f52c745c8cd14
local       dc07dfb191bb2d879e618e11effaacd0e60cc092d590fe0e911def2f655afe74
local       e652f37f853f7e8731008ceba6fe39616efc86648c8c42a1386c8095fcc0461
local       ee23959ca129d062a834303e8e7de7b9d2dc138810b6443ce3626e56fa3fe3d6
local       jenkins_data
local       k3d-mycluster-images
local       lab-10_sonar_data
local       lab-10_sonar_db_data
local       lab-10_sonar-extensions
local       my-app_jenkins_home
local       update
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/ff/Containerization and DevOps/Containerization And DevOps_Lab/Lab-5$ docker inspect web1 | grep -A 5 Mounts
      "Mounts": [
        {
          "Type": "volume",
          "Name": "aee9db2b4cab8655472fc32d93dada40d38f324d8e0b1ee08233b47ad60b5528",
          "Source": "/var/lib/docker/volumes/aee9db2b4cab8655472fc32d93dada40d38f324d8e0b1ee08233b47ad60b5528/_data",
          "Destination": "/app/data",
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/ff/Containerization and DevOps/Containerization And DevOps_Lab/Lab-5$
```

2. Named Volumes

```
# Create named volume
docker volume create mydata

# Use named volume
docker run -d -v mydata:/app/data --name web2 nginx

# List volumes
docker volume ls
# Shows: mydata

# Inspect volume
docker volume inspect mydata
```

3. Bind Mounts (Host Directory)

```
# Create directory on host
mkdir ~/myapp-data

# Mount host directory to container
docker run -d -v ~/myapp-data:/app/data --name web3 nginx

# Add file on host
echo "From Host" > ~/myapp-data/host-file.txt

# Check in container
docker exec web3 cat /app/data/host-file.txt

# Shows: From Host
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$ docker run -d -v ~/myapp-data:/app/data --name web3 nginx
bf95c892b8c50f884d8efbcc3dedcf50ff541089bc4cb089cf747581e1dda88c
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$ echo "From Host" > ~/myapp-data/host-file.txt
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$ docker exec web3 cat /app/data/host-file.txt
From Host
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$
```

Lab 3: Practical Volume Examples

Example 1: Database with Persistent Storage

```
# MySQL with named volume
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0

# Check data persists
docker stop mysql-db
docker rm mysql-db

# New container with same volume
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0

# Data is preserved!
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
bb5107df7baa: Pulling fs layer
62f1cc0a64ca: Download complete
4f7cfd23b111: Download complete
c689d1836928: Waiting
caa784df8611: Waiting
fe997c72f3e9: Waiting
38fe0e0986de: Waiting
09b552d72653: Waiting
a0db31010df3: Waiting
b723face7585: Waiting
139627f60883: Waiting
```

Example 2: Web App with Configuration Files

```
# Create config directory
mkdir ~/nginx-config

# Create nginx config file
echo 'server {
    listen 80;
    server_name localhost;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf

# Run nginx with config bind mount
docker run -d \
    --name nginx-custom \
    -p 8080:80 \
    -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
    nginx

# Test
curl http://localhost:8080
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraizaiton And DEvops_Lab/Lab/Lab-5$ echo 'server {
listen 80;
server_name localhost;
location / {
    return 200 "Hello from mounted config!";
}
}' > ~/nginx-config/nginx.conf
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraizaiton And DEvops_Lab/Lab/Lab-5$ docker run -d \
--name nginx-custom \
-p 8080:80 \
-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx
dcd89a387fc8951c62240ad1bd4273a191b3753eb61bb3dc119a4ac1af8146bc
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraizaiton And DEvops_Lab/Lab/Lab-5$ curl http://localhost:8080
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraizaiton And DEvops_Lab/Lab/Lab-5$
```

Lab 4: Volume Management Commands

```
# List all volumes
docker volume ls

# Create a volume
docker volume create app-volume

# Inspect volume details
docker volume inspect app-volume

# Remove unused volumes
docker volume prune

# Remove specific volume
docker volume rm volume-name

# Copy files to/from volume
docker cp local-file.txt container-name:/path/in/volume
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-5$ docker volume create app-volume
app-volume
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-5$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-03-04T11:07:18Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-5$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
551c22f645cc7324a47830fa173baaa0dc272c433a83a9ced999321ecd645848
dc07dfb191bb2d879e618e11effaacd0e60cc092d590fe0e911def2f655afe74
5712fe64ba49e3596ed5e5cabcd6d3b7dd890be0cca49a0c7caeff313c9429a85
c76d238275c9d4701de6da5b032520fb1269ac94b8ad2386f6e330ea2fa989e9
7b771ce4e592935b9d1e5ef6f336ef669148c94777d921fd589b4acff31ebee
087c3a07dc435d48bd89129e6a146d77f80c429ef024870624c779f93ff7dff
1c1b3d5ea8bd1d7a926c05ff1b50dc28d8a2b99473992631d231dbc545e6d414
457d9ec51623465f9f1dafeb412b4e01cd/d14ffc810eae776ea0c7bcb9a06e5
d6e00f7334f0/acc23e92ab4b6f6425844511735c6c24d75234f52c745c8cd14
ee23959ca129d062a834303e8e7de7b9d2dc138810b6443ce3626e56fa3fe3d6
9e21c77f884ef3d1c552214f8687da8d252fd0699e9a11c7a245382f7436028f

Total reclaimed space: 1.459GB
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-5$
```

Part 2: Environment Variables

Lab 1: Setting Environment Variables

Method 1: Using -e flag

```
# Single variable
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  my-node-app

# Multiple variables
docker run -d \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  my-app
```

Method 2: Using --env-file

```
# Create .env file
echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env

# Use env file
docker run -d \
  --env-file .env \
  --name app2 \
  my-app

# Use multiple env files
docker run -d \
  --env-file .env \
  --env-file .env.secrets \
  my-app
```

Method 3: In Dockerfile

```
# Set default environment variables
ENV NODE_ENV=production
ENV PORT=3000
ENV APP_VERSION=1.0.0

# Can be overridden at runtime
```

Lab 2: Environment Variables in Applications

Python Flask Example

```
# app.py
import os
from flask import Flask

app = Flask(__name__)

# Read environment variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```


Dockerfile with Environment Variables

```
FROM python:3.9-slim

# Set environment variables at build time
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

# Default runtime environment variables
ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
```

```
divyanshuabuntug@DESKTOP-GVHQ9HP: /mnt/r/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-5$ docker build -t envpython .
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory
```

```
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
```

```
Sending build context to Docker daemon  1.347MB
Step 1/11 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/11 : ENV PYTHONUNBUFFERED=1
--> Running in 148e8f15aacd
--> Removed intermediate container 148e8f15aacd
--> b2e162f2fc16
Step 3/11 : ENV PYTHONDONTWRITEBYTECODE=1
--> Running in 43eb3c60109d
--> Removed intermediate container 43eb3c60109d
--> 28f177251690
Step 4/11 : WORKDIR /app
--> Running in d9bae9900e9f
--> Removed intermediate container d9bae9900e9f
--> b15c45fb3e7
```

Experiment 6

Title: Comparison of Docker Run and Docker Compose

PART A – THEORY

1. Objective

To understand the relationship between `docker run` and Docker Compose, and to compare their configuration syntax and use cases.

2. Background Theory

2.1 Docker Run (Imperative Approach)

The `docker run` command is used to create and start a container from an image. It requires explicit flags for:

- Port mapping (`-p`)
- Volume mounting (`-v`)
- Environment variables (`-e`)
- Network configuration (`--network`)
- Restart policies (`--restart`)
- Resource limits (`--memory` , `--cpus`)
- Container name (`--name`)

This approach is **imperative**, meaning you specify step-by-step instructions.

Example:

```
docker run -d \
  --name my-nginx \
  -p 8080:80 \
  -v ./html:/usr/share/nginx/html \
  -e NGINX_HOST=localhost \
  --restart unless-stopped \
  nginx:alpine
```

2.2 Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes in a structured format.

Instead of multiple `docker run` commands, a single command is used:

```
docker compose up -d
```

Compose is **declarative**, meaning you define the desired state of the application.

Equivalent Compose file:

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    environment:
      NGINX_HOST: localhost
    restart: unless-stopped
```

```
divyanshu@ubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part1$ docker compose up -d
WARN[0000] /mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part1/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 2/2
 ✓ Network part1_default Created                                0.1s
 ✓ Container my-nginx Started                                  0.4s
```

3. Mapping: Docker Run vs Docker Compose

Docker Run Flag	Docker Compose Equivalent
<code>-p 8080:80</code>	<code>ports:</code>
<code>-v host:container</code>	<code>volumes:</code>
<code>-e KEY=value</code>	<code>environment:</code>
<code>--name</code>	<code>container_name:</code>
<code>--network</code>	<code>networks:</code>
<code>--restart</code>	<code>restart:</code>
<code>--memory</code>	<code>deploy.resources.limits.memory</code>
<code>--cpus</code>	<code>deploy.resources.limits.cpus</code>
<code>-d</code>	<code>docker compose up -d</code>

4. Advantages of Docker Compose

1. Simplifies multi-container applications
2. Provides reproducibility
3. Version controllable configuration
4. Unified lifecycle management
5. Supports service scaling

Example:

```
docker compose-up --scale web=3
```

PART B – PRACTICAL TASK

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

Execute:

```
docker run -d \  
  --name lab-nginx \  
  -p 8081:80 \  
  -v $(pwd)/html:/usr/share/nginx/html \  
  nginx:alpine
```

Verify:

```
docker ps
```

Access:

```
http://localhost:8081
```

Stop and remove container:

```
docker stop lab-nginx  
docker rm lab-nginx
```

Step 2: Run Same Setup Using Docker Compose

Create `docker-compose.yml` :

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Run:

```
docker compose up -d
```

Verify:

```
docker compose ps
```

Stop:

```
docker compose down
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab/Lab/Lab-6/Part1$ docker-compose ps
Name                Command             State      Ports
-----
my-nginx            /docker-entrypoint.sh nginx ... Up          0.0.0.0:8080->80/tcp, :::8080->80/tcp
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab/Lab/Lab-6/Part1$ docker-compose down
Stopping my-nginx ... done
Removing my-nginx ... done
Removing network part1_default
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab/Lab/Lab-6/Part1$
```

Task 2: Multi-Container Application

Objective:

Deploy WordPress with MySQL using:

1. Docker Run (manual way)
2. Docker Compose (structured way)

A. Using Docker Run

1. Create network:

```
docker network create wp-net
```

1. Run MySQL:

```
docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
```

1. Run WordPress:

```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```

Test:

```
http://localhost:8082
```

```

divyanshuubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part1$ docker network create wp-net
9c905fc6ea524a4eeecfd4d1b0ebb35ea7109f889751c7263de8a24660bf4b6
divyanshuubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part1$ docker run -d \
--name mysql \
--network wp-net \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
9851adc0c79121e8989c603cf93f8edcd71e2ff417c9e761b5e0496d163e07c
divyanshuubuntu@DESKTOP-GVHQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part1$ docker run -d \
--name wordpress \
--network wp-net \
-p 8082:80 \
-e WORDPRESS_DB_HOST=mysql \
-e WORDPRESS_DB_PASSWORD=secret \
wordpress:latest
0ddf6203bd9eec3bde51347d0f7f159d4358dae27fcee016d7073e141df50dfc

```

B. Using Docker Compose

Create `docker-compose.yml` :

```

version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:

```


Run:

```
docker compose-up -d
```

Stop:

```
docker compose-down -v
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/Part3$ docker compose up -d
WARN[0000] /mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/Part3/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/4
  ✓ Container part3_mysql_1      Recreated          2.0s
  ✓ Container 4a0a6708aceb_part3_wordpress_1 Recreated          0.1s
  ✓ Container part3_mysql-1      Started          0.2s
  ⚙ Container part3-wordpress-1   Starting          0.3s
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint part3-wordpress-1 (dd3e722bee6abc8ec7e9ce1b59953ed030a4a8a4138feeb393ba64bef93ece3): Bind for 0.0.0.0:8082 failed: port is already allocated
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/Part3$ docker compose down -v
WARN[0000] /mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/Part3/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  ✓ Container part3-wordpress-1   Removed          0.0s
  ✓ Container part3_mysql-1      Removed          2.1s
  ✓ Volume part3_mysql_data      Removed          0.0s
  ✓ Network part3_default        Removed          0.4s
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/Part3$
```

PART C – CONVERSION & BUILD-BASED TASKS

Task 3: Convert Docker Run to Docker Compose

Problem 1: Basic Web Application

Given Docker Run Command:

```
docker run -d \  
  --name webapp \  
  -p 5000:5000 \  
  -e APP_ENV=production \  
  -e DEBUG=false \  
  --restart unless-stopped \  
  node:18-alpine
```

Student Task:

1. Write an equivalent `docker-compose.yml`
2. Ensure:
 - Same container name
 - Same port mapping
 - Same environment variables
 - Same restart policy

3. Run using:

```
docker compose up -d
```

4. Verify using:

```
docker compose ps
```

Solution:

docker-compose file:

```
version: '3.8'

services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contrarization And Devops_Lab/Lab-6/Problem1$ docker compose up -d
WARN[0000] /mnt/f/Contrarization and DevOps/Contrarization And Devops_Lab/Lab-6/Problem1/docker-compose.yml: the attribute 'version' is obsolete, it will be i
gnored, please remove it to avoid potential confusion
[+] Running 5/5
✔ webapp Pulled                                                                                                                                           21.4s
✔ f18232174bc9 Pull complete                                                                                                                           4.6s
✔ dd71dde834b5 Pull complete                                                                                                                           16.2s
✔ 1e5a4c89cee5 Pull complete                                                                                                                           16.3s
✔ 25ff2d83641 Pull complete                                                                                                                           16.3s
[+] Running 2/2
✔ Network problem1_default Created                                                                                                                    0.1s
✔ Container webapp Started                                                                                                                            0.9s
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contrarization And Devops_Lab/Lab-6/Problem1$ docker compose ps
WARN[0000] /mnt/f/Contrarization and DevOps/Contrarization And Devops_Lab/Lab-6/Problem1/docker-compose.yml: the attribute 'version' is obsolete, it will be i
gnored, please remove it to avoid potential confusion
NAME        IMAGE          COMMAND                  SERVICE    CREATED        STATUS               PORTS
webapp      node:18-alpine "docker-entrypoint.s..." webapp     6 seconds ago  Restarting (0) less than a second ago
```

Problem 2: Volume + Network Configuration

Given Docker Run Commands:

```
docker network create app-net
```

```
docker run -d \  
  --name postgres-db \  
  --network app-net \  
  -e POSTGRES_USER=admin \  
  -e POSTGRES_PASSWORD=secret \  
  -v pgdata:/var/lib/postgresql/data \  
  postgres:15
```

```
docker run -d \  
  --name backend \  
  --network app-net \  
  -p 8000:8000 \  
  -e DB_HOST=postgres-db \  
  -e DB_USER=admin \  
  -e DB_PASS=secret \  
  python:3.11-slim
```

Student Task:

1. Create a single `docker-compose.yml` file that:
 - Defines both services
 - Creates named volume `pgdata`
 - Creates custom network `app-net`
 - Uses `depends_on`
2. Bring up services using one command.
3. Stop and remove everything properly.

Solution:

docker-compose file:

```
version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:
      - postgres-db
    networks:
      - app-net

volumes:
  pgdata:

networks:
  app-net:
```

```

divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-6/Problem2$ docker compose up -d
WARN[0000] /mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-6/Problem2/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 19/19
  ✓ postgres-db Pulled                    53.6s
  ✓ backend Pulled                       18.0s

[+] Running 4/4
  ✓ Network problem2_app-net Created      0.1s
  ✓ Volume problem2_pgdata Created        0.0s
  ✓ Container postgres-db Started         0.7s
  ✓ Container backend Started             0.6s
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-6/Problem2$ docker compose ps
WARN[0000] /mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-6/Problem2/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE              COMMAND                  SERVICE    CREATED          STATUS          PORTS
postgres-db         postgres:15        "docker-entrypoint.s..." postgres-db 11 seconds ago   Up 10 seconds   5432/tcp
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-6/Problem2$ docker compose down -v
WARN[0000] /mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-6/Problem2/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  ✓ Container backend Removed            0.0s
  ✓ Container postgres-db Removed        0.3s
  ✓ Network problem2_app-net Removed     0.4s
  ✓ Volume problem2_pgdata Removed       0.0s
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOps/Contratization And Devops_Lab/Lab/Lab-6/Problem2$

```

Task 4: Resource Limits Conversion

Given Docker Run Command:

```

docker run -d \
  --name limited-app \
  -p 9000:9000 \
  --memory="256m" \
  --cpus="0.5" \
  --restart always \
  nginx:alpine

```

Student Task:

1. Convert this to Docker Compose.
2. Add resource limits using:

```
deploy:
  resources:
    limits:
```

3. Explain:

- When `deploy` works
- Difference between normal Compose mode and Swarm mode

Solution:

docker-compose file:

```
version: '3.8'

services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always

    deploy:
      resources:
        limits:
          memory: 256M
          cpus: "0.5"
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOps/Contraization And DEvops_Lab/Lab-6/Problem3$ docker compose up -d
WARN[0000] /mnt/f/Contarization and DevOps/Contraization And DEvops_Lab/Lab-6/Problem3/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 2/2
 ✓ Network problem3 default Created 0.1s
 ✓ Container limited-app Started 0.3s
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOps/Contraization And DEvops_Lab/Lab-6/Problem3$ docker compose ps
WARN[0000] /mnt/f/Contarization and DevOps/Contraization And DEvops_Lab/Lab-6/Problem3/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME          IMAGE          COMMAND                  SERVICE    CREATED        STATUS        PORTS
limited-app    nginx:alpine   "/docker-entrypoint..." limited-app  9 seconds ago  Up 8 seconds  80/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOps/Contraization And DEvops_Lab/Lab-6/Problem3$ docker compose down -v
WARN[0000] /mnt/f/Contarization and DevOps/Contraization And DEvops_Lab/Lab-6/Problem3/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 2/2
 ✓ Container limited-app Removed 0.5s
 ✓ Network problem3 default Removed 0.3s
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOps/Contraization And DEvops_Lab/Lab-6/Problem3$
```

When `deploy` Works

- The `deploy` section works only in Docker Swarm mode.
- It is applied when using: `bash docker stack deploy`

Difference: Compose Mode vs Swarm Mode

Normal Compose Mode

- Used for local development
- Single-node only
- `deploy` section is ignored

Run using:

```
docker compose up
```

Swarm Mode

- Used for production and clustering
- Supports multi-node setup
- Supports scaling and rolling updates
- `deploy` section works

Run using:

```
docker stack deploy
```

PART D – USING DOCKERFILE INSTEAD OF STANDARD IMAGE

Now increase difficulty.

Instead of using prebuilt images like:

```
image: node:18-alpine
```

Students must:

- Create their own Dockerfile
- Build image using Compose
- Run container from that build

Task 5: Replace Standard Image with Dockerfile (Node App)

Scenario

You are given:

```
docker run -d -p 3000:3000 node:18-alpine
```

Now instead of directly using `node:18-alpine`, you must:

1. Create a simple Node.js app
 2. Write a Dockerfile
 3. Use Docker Compose `build:` option
-

Step 1: Create `app.js`

```
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```

Step 2: Create `Dockerfile`

```
FROM node:18-alpine

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step 3: Create `docker-compose.yml`

```
version: '3.8'

services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-6/Part4$ docker compose up --build -d
WARN[0000] /mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-6/Part4/docker-compose.yml: the attribute `version` is obsolete, it w
red, please remove it to avoid potential confusion
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 0.5s (9/9) FINISHED
=> [nodeapp internal] load build definition from Dockerfile
=> => transferring dockerfile: 132B
=> [nodeapp internal] load metadata for docker.io/library/node:18-alpine
=> [nodeapp internal] load .dockerignore
=> => transferring context: 2B
=> [nodeapp 1/3] FROM docker.io/library/node:18-alpine
=> [nodeapp internal] load build context
=> => transferring context: 179B
=> [nodeapp 2/3] WORKDIR /app
=> [nodeapp 3/3] COPY app.js .
=> [nodeapp] exporting to image
=> => exporting layers
=> => writing image sha256:0144556e1d25459afa7722a8d88dfb9639095753c7aea832abaf6d6ad5fbbd67
=> => naming to docker.io/library/part4-nodeapp
=> [nodeapp] resolving provenance for metadata file
[+] Running 3/3
 ✓ nodeapp Built
 ✓ Network part4_default Created
 ✓ Container custom-node-app Started
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-6/Part4$ http://localhost:3000
-bash: http://localhost:3000: No such file or directory
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-6/Part4$ curl http://localhost:3000
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab-6/Part4$
```

Student Task:

1. Build and run using:

```
docker compose up --build -d
```

2. Verify in browser:

```
http://localhost:3000
```

3. Modify `app.js` message.
4. Rebuild and observe changes.

```

divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part4$ curl http://localhost:3000
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part4$ docker compose up --build -d
WARN[0000] /mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part4/docker-compose.yml: the attribute `version` is obsolete, it w
red, please remove it to avoid potential confusion
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 0.2s (9/9) FINISHED
=> [nodeapp internal] load build definition from Dockerfile
=> => transferring dockerfile: 132B
=> [nodeapp internal] load metadata for docker.io/library/node:18-alpine
=> [nodeapp internal] load .dockerignore
=> => transferring context: 2B
=> [nodeapp 1/3] FROM docker.io/library/node:18-alpine
=> [nodeapp internal] load build context
=> => transferring context: 28B
=> CACHED [nodeapp 2/3] WORKDIR /app
=> CACHED [nodeapp 3/3] COPY app.js .
=> [nodeapp] exporting to image
=> => exporting layers
=> => writing image sha256:0144556e1d25459afa7722a8d88dfb9639095753c7aea832abafed6ad5fbbd67
=> => naming to docker.io/library/part4-nodeapp
=> [nodeapp] resolving provenance for metadata file
[+] Running 2/2
✔ nodeapp Built
✔ Container custom-node-app Running
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part4$ curl http://localhost:3000
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab-6/Part4$ curl http://localhost:3000

```

Advanced Build Challenge

Task 6: Multi-Stage Dockerfile with Compose

Requirement: [🔗](#)

Create a simple Python FastAPI or Node production-ready app using:

- Multi-stage Dockerfile
- Smaller final image
- Use Compose to build it

Must:

1. Write multi-stage Dockerfile
2. Use `build:` in Compose
3. Add environment variables
4. Add volume mount for development mode
5. Compare image size:

```
docker images
```

Solution:

DockerFile:

```
# ----- Stage 1: Builder -----
FROM python:3.11-slim AS builder

WORKDIR /app

COPY app/requirements.txt .

RUN pip install --upgrade pip \
    && pip install --prefix=/install -r requirements.txt

# ----- Stage 2: Production -----
FROM python:3.11-slim

WORKDIR /app

COPY --from=builder /install /usr/local
COPY app/ .

ENV PYTHONUNBUFFERED=1

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose file:

```
version: "3.8"
```

docker-compose file:

```
version: "3.8"

services:
  fastapi-app:
    build: .
    container_name: fastapi-app
    ports:
      - "8000:8000"
    environment:
      APP_ENV: production
    volumes:
      - ./app:/app # For development mode (auto reload if used)
    restart: unless-stopped
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6$ cd AdvancedTask
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/AdvancedTask$ docker compose up --build -d
WARN[0000] /mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/AdvancedTask/docker-compose.yml: the attribute `version` is obsolete, it will
be ignored, please remove it to avoid potential confusion
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 12.0s (12/12) FINISHED                                docker:default
=> [fastapi-app internal] load build definition from Dockerfile      0.1s
=> => transferring dockerfile: 508B                                  0.0s
=> [fastapi-app internal] load metadata for docker.io/library/python:3.11-slim 0.0s
=> [fastapi-app internal] load .dockerignore                        0.0s
=> => transferring context: 2B                                         0.0s
=> [fastapi-app internal] load build context                        0.1s
=> => transferring context: 992B                                       0.1s
=> [fastapi-app builder 1/4] FROM docker.io/library/python:3.11-slim 0.1s
=> [fastapi-app builder 2/4] WORKDIR /app                          0.1s
=> [fastapi-app builder 3/4] COPY app/requirements.txt .           0.0s
=> [fastapi-app builder 4/4] RUN pip install --upgrade pip && pip install --prefix=/install -r requirements.txt 10.5s
=> [fastapi-app stage-1 3/4] COPY --from=builder /install /usr/local 0.4s
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/AdvancedTask$ curl http://localhost:8000
{"message":"Hello from FastAPI!", "env":"production"}divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contrarization And Devops_Lab/Lab-6/AdvancedTask$ docker images
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
advancedtask-fastapi-app  latest      28ebb41820e5   About a minute ago   140MB
part4-nodeapp        latest      0144556e1d25   8 minutes ago     127MB
<none>               <none>      c86f1ace990f   2 hours ago       122MB
python               3.11-slim   7c68b5683872   16 hours ago       124MB
```

Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

Course: DevOps	Student: Divyanshu Singh	SAP ID: 500122856	University: UPES, Dehradun
----------------	--------------------------	-------------------	----------------------------

Table of Contents

- 1. Aim
- 2. Objectives
- 3. Theory
- 4. Prerequisites
- 5. Part A – GitHub Repository Setup
- 6. Part B – Jenkins Setup using Docker
- 7. Part C – Jenkins Configuration
- 8. Part D – GitHub Webhook Integration
- 9. Part E – Execution Flow
- 10. Screenshots
- 11. Observations
- 12. Result

Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application + Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build & push process using webhook triggers
- Use same host (Docker) as Jenkins agent

Theory

What is Jenkins?

Jenkins is a **web-based GUI automation server** used to build applications, test code, and deploy software. It provides:

- A browser-based dashboard
- A rich plugin ecosystem (GitHub, Docker, etc.)
- Pipeline as Code using a `Jenkinsfile`

What is CI/CD?

Term	Definition
Continuous Integration (CI)	Code is automatically built and tested after each commit
Continuous Deployment (CD)	Built artifacts (Docker images) are automatically delivered/deployed

Workflow Overview

Developer → GitHub → Webhook → Jenkins → Build → Docker Hub

Prerequisites

- Docker & Docker Compose installed
- GitHub account
- Docker Hub account
- Basic Linux command knowledge

Part A – GitHub Repository Setup

5.1 Create Repository

Create a repository on GitHub named `my-app`.

5.2 Project Structure

```
my-app/  
├── app.py  
├── requirements.txt  
├── Dockerfile  
└── Jenkinsfile
```

5.3 Application Code

`app.py`

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "Hello from CI/CD Pipeline! DIVYANSHU SINGH is here!\n Change: Checking if the pipeline is working fine."  
    #return "Hello from CI/CD Pipeline!, my sapid is 500122856"  
  
app.run(host="0.0.0.0", port=80)
```

`requirements.txt`

5.4 Dockerfile

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

Build Process:

1. Source code is pushed to GitHub
2. Jenkins pulls the code
3. Dockerfile creates the environment, installs dependencies, and packages the app
4. Output → Docker Image

5.5 Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "divyanshusingh151103/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {
                git branch: 'main', url: 'https://github.com/divyanshusingh1511/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
```

```
stage('Login to Docker Hub') {
    steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
            sh 'echo $DOCKER_TOKEN | docker login -u divyanshusingh151103 --password-stdin'
        }
    }
}

stage('Push to Docker Hub') {
    steps {
        sh 'docker push $IMAGE_NAME:latest'
    }
}
}
```

Part B – Jenkins Setup using Docker

6.1 Docker Compose File

```
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

6.2 Start Jenkins

```
docker-compose up -d
```

Access Jenkins at: `http://localhost:8080`

6.3 Unlock Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

6.4 Initial Setup

1. Install suggested plugins
2. Create admin user

Part C – Jenkins Configuration

7.1 Add Docker Hub Credentials

Navigate to:

Manage Jenkins → Credentials → Add Credentials

Field	Value
Type	Secret Text
ID	<code>dockerhub-token</code>
Value	Docker Hub Access Token

7.2 Create Pipeline Job

1. New Item → Pipeline
2. Name: `ci-cd-pipeline`
3. Configure: Pipeline script from SCM
 - SCM: Git
 - Repo URL: your GitHub repo URL
 - Script Path: `Jenkinsfile`

Part D – GitHub Webhook Integration

8.1 Configure Webhook

In GitHub, go to:

Settings → Webhooks → Add Webhook

Field	Value
Payload URL	<code>http://<your-server-ip>:8080/github-webhook/</code>
Content Type	<code>application/json</code>
Events	Push events

Part E – Execution Flow

Stage 1: Code Push	→ Developer updates code in GitHub
Stage 2: Webhook Trigger	→ GitHub sends push event to Jenkins
Stage 3: Clone	→ Jenkins pulls latest code from GitHub
Stage 4: Build	→ Docker builds image using Dockerfile
Stage 5: Auth	→ Jenkins logs into Docker Hub using stored token
Stage 6: Push	→ Image pushed to Docker Hub
Stage 7: Artifact Ready	→ Docker image available globally

Role of Same Host Agent

Jenkins runs inside Docker with the Docker socket mounted:

`/var/run/docker.sock`

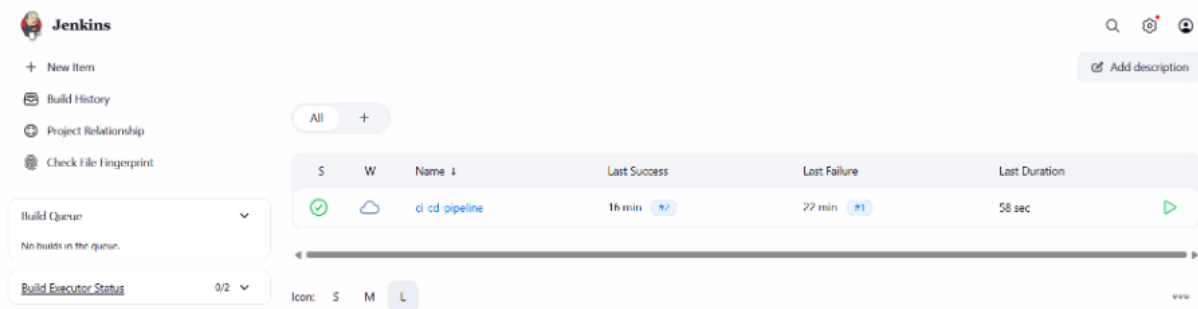
This allows Jenkins to directly control the host Docker daemon — building and pushing images without a separate agent.

Screenshots

Instructions: Replace each placeholder below with your actual screenshot.

To add a screenshot: save the image in the same folder as this README, then update the path.

1. Jenkins Dashboard



2. Docker Compose Up – Jenkins Container Running

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contratization And Devops_Lab/Lab/Lab-7/my-app/my-app$ docker-compose up -d
Creating network "my-app_default" with the default driver
Creating volume "my-app_jenkins_home" with default driver
Creating jenkins ... done
```

3. Jenkins Initial Unlock Screen

Getting Started

Unlock Jenkins

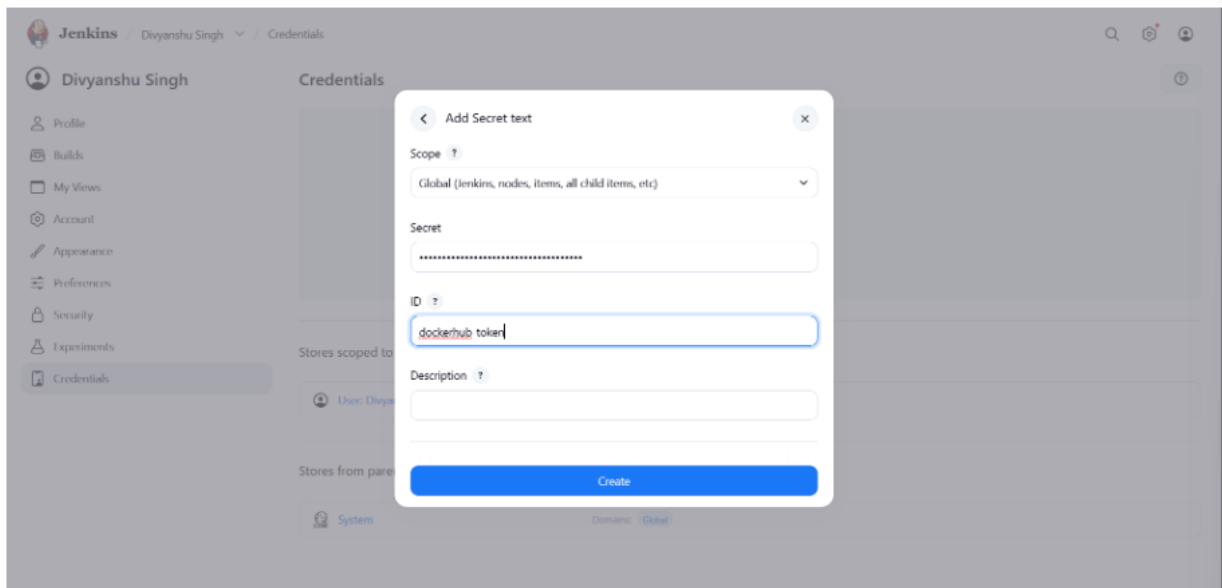
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

4. Adding Docker Hub Credentials in Jenkins



5. Pipeline Job Configuration (SCM Settings)

 Jenkins

ci-cd-pipeline

Configure

Search

Help

Logout

Configure

General

Triggers

Pipeline

Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/divyanshusingh1511/my-app.git

Credentials

- none -

+ Add

Jenkins

ci-cd-pipeline

Configure

Search

Help

Logout

Configure

General

Triggers

Pipeline

Advanced

Branches to build

Branch Specifier (blank for 'any')

*/master

+ Add Branch

Repository browser

(Auto)

Additional Behaviours

+ Add

Script Path

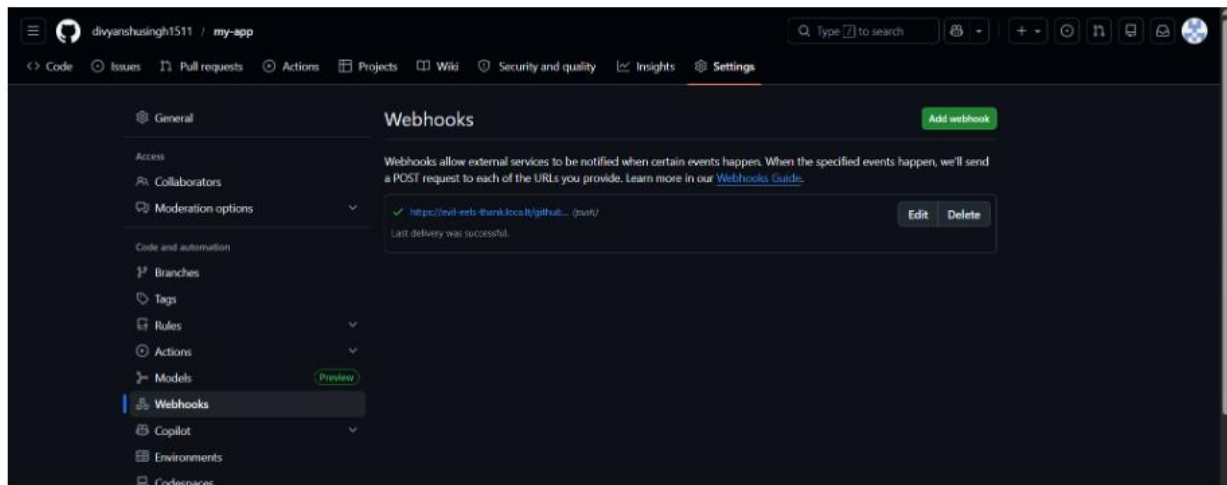
Jenkinsfile

☒ Lightweight checkout

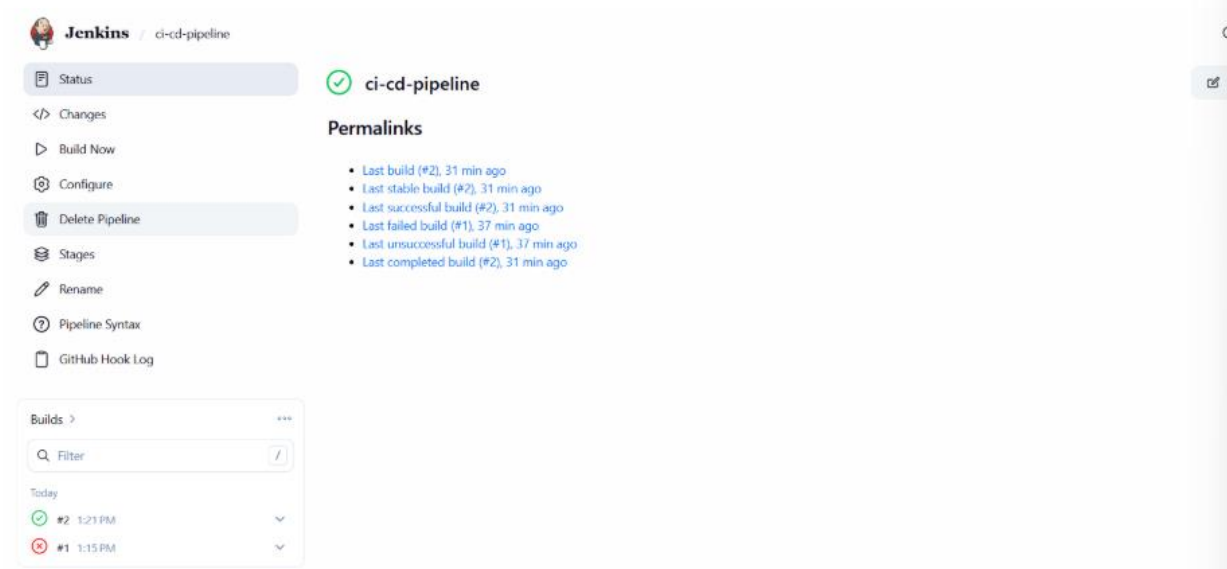
Save

Apply

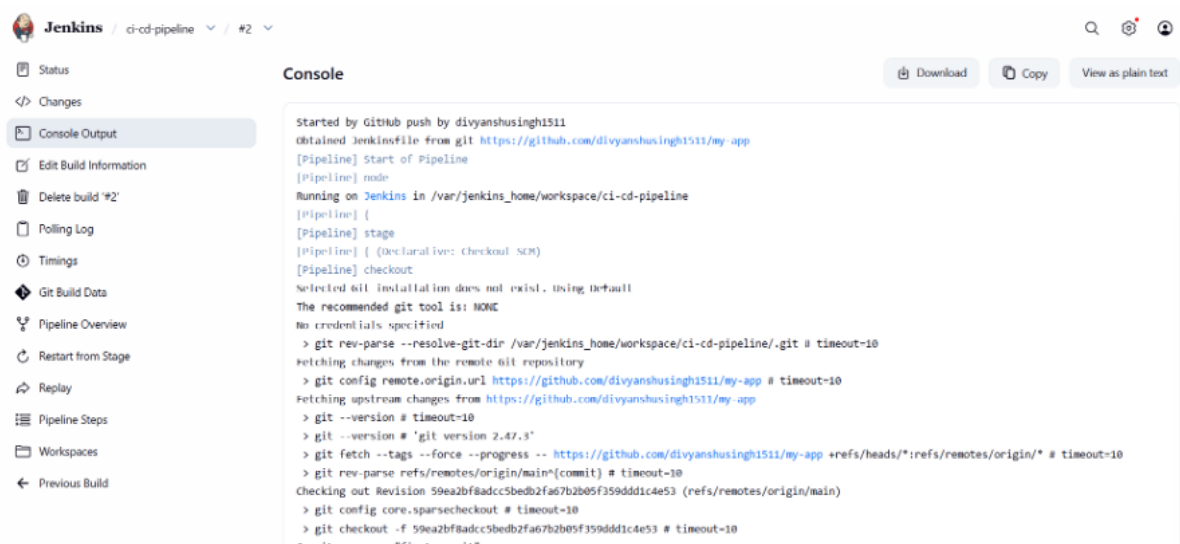
6. GitHub Webhook Configuration



7. Jenkins Pipeline Execution – Stage View



8. Console Output – Successful Build & Push



The screenshot shows the Jenkins web interface for a build named 'ci-cd-pipeline' with build number '#2'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#2', Polling Log, Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main console area displays the following output:

```
Started by Github push by divyanshusingh1511
Obtained Jenkinsfile from git https://github.com/divyanshusingh1511/my-app
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/ci-cd-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: checkout SCM)
[Pipeline] checkout
Selected git installation does not exist, using default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/ci-cd-pipeline/.git # timeout=10
Fetching changes from the remote git repository
> git config remote.origin.url https://github.com/divyanshusingh1511/my-app # timeout=10
Fetching upstream changes from https://github.com/divyanshusingh1511/my-app
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --tags --force --progress -- https://github.com/divyanshusingh1511/my-app +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 59ea2bf8adcc5bedb2fa67b2b05f359ddd1c4e53 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 59ea2bf8adcc5bedb2fa67b2b05f359ddd1c4e53 # timeout=10
..
```

Tip: Create a `screenshots/` folder in your repo and drop all images there.
Then remove the triple backticks (`````) around each image tag to make them render properly.

Result

Successfully implemented a complete CI/CD pipeline where:

- Source code and pipeline definition are maintained in GitHub
- Jenkins automatically detects changes via webhook triggers
- Docker image is built directly on the host agent
- Image is securely pushed to Docker Hub using stored credentials

Experiment 9 – Ansible Automation Using Docker Servers

Overview

This experiment demonstrates how **Ansible** can be used to automate configuration and management of multiple servers. Docker containers are used as test servers, and Ansible connects to them via SSH to execute tasks defined in a playbook.

Objective

- Install and configure Ansible.
 - Create Docker containers to act as servers.
 - Establish SSH key-based authentication.
 - Create an Ansible inventory.
 - Execute Ansible commands and playbooks to configure servers automatically.
-

Prerequisites

- Ubuntu/WSL environment
 - Docker installed
 - Python installed
 - Internet connection
-

Step 1: Install Ansible

Update packages and install Ansible.

```
sudo apt update -y
sudo apt install ansible -y
```

Verify installation:

```
ansible --version
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab/Lab/Lab-9$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/divyanshubuntu/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/divyanshubuntu/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab/Lab/Lab-9$
```

Test Ansible locally:

```
ansible localhost -m ping
```

Expected Output:

```
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab/Lab/Lab-9$ ansible localhost -m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab/Lab/Lab-9$
```

Step 2: Generate SSH Key Pair

Create SSH keys for authentication between control node and servers.

```
ssh-keygen -t rsa -b 4096
```

Copy the keys to the working directory:

```
cp ~/.ssh/id_rsa .  
cp ~/.ssh/id_rsa.pub .
```

Step 3: Create Dockerfile for SSH Server

Create a file named `Dockerfile`.

```
nano Dockerfile
```

Paste the following configuration:

```
FROM ubuntu  
  
RUN apt update -y  
RUN apt install -y python3 python3-pip openssh-server  
RUN mkdir -p /var/run/sshd  
  
RUN mkdir -p /run/sshd && \  
  echo 'root:password' | chpasswd && \  
  sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \  
  sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \  
  sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config  
  
RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh  
  
COPY id_rsa /root/.ssh/id_rsa  
COPY id_rsa.pub /root/.ssh/authorized_keys  
  
RUN chmod 600 /root/.ssh/id_rsa  
RUN chmod 644 /root/.ssh/authorized_keys
```

```
RUN chmod 600 /root/.ssh/id_rsa
RUN chmod 644 /root/.ssh/authorized_keys

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
```

Step 4: Build Docker Image

Build the Docker image named `ubuntu-server`.

```
docker build -t ubuntu-server .
```

Step 5: Start Multiple Docker Servers

Run four containers that will act as servers.

```
for i in {1..4}; do
  docker run -d -p 220${i}:22 --name server${i} ubuntu-server
done
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-9$ for i in {1..4}; do
  docker run -d -p 220${i}:22 --name server${i} ubuntu-server
done
5ab3b97ea85f7240bd57add086398ee8088269c77661ddbe2a67a2e55e4e12dd
2ac0f5dd31bfa4e1dd0e4c848a07ef27c41ac8634fe8896048c68defa00a334f
a4d20590314290713845efcb90df8fd35d4bed1a2d880634bf72a9d59716ce
6c06b2b4eba66ba39748ab6ffce074c7df3b9e462557f03dbf7d9659c5c4784c
```

Verify running containers:

```
docker ps
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-9$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS	PORTS
6c06b2b4eba6	ubuntu-server	"/usr/sbin/sshd -D"	server4	14 seconds ago	Up 14 seconds	0.0.0.0:2204->22/tcp, [::]:2204->22/tcp
a4d205903142	ubuntu-server	"/usr/sbin/sshd -D"	server3	14 seconds ago	Up 14 seconds	0.0.0.0:2203->22/tcp, [::]:2203->22/tcp
2ac0f5dd31bf	ubuntu-server	"/usr/sbin/sshd -D"	server2	15 seconds ago	Up 14 seconds	0.0.0.0:2202->22/tcp, [::]:2202->22/tcp
5ab3b9/ea85f	ubuntu-server	"/usr/sbin/sshd -D"	server1	15 seconds ago	Up 15 seconds	0.0.0.0:2201->22/tcp, [::]:2201->22/tcp
b1a77bec2779	jenkins/jenkins:its	"/usr/bin/tini -- /u..."		4 hours ago	Up 27 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp

Step 6: Obtain Container IP Addresses

Retrieve the IP address of each container.

```
docker inspect -f '{{.IPAddr}}' server1
docker inspect -f '{{.IPAddr}}' server2
docker inspect -f '{{.IPAddr}}' server3
docker inspect -f '{{.IPAddr}}' server4
```

Example output:

```
172.17.0.2
172.17.0.3
172.17.0.4
172.17.0.5
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-9$ docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddr}}{{end}}' server1
172.17.0.2
172.17.0.3
172.17.0.4
172.17.0.5
```

Step 7: Test SSH Connectivity

Verify SSH access to a container.

```
ssh -i ~/.ssh/id_rsa root@172.17.0.3
```

Exit after successful login:

```
exit
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contrarization And Devops_Lab/Lab/Lab-9$ ssh -i ~/.ssh/id_rsa root@172.17.0.3
The authenticity of host '172.17.0.3 (172.17.0.3)' can't be established.
ED25519 key fingerprint is SHA256:JhyslIge1U080pNu522CU9dDKTSVYJF82HfYuoOKQqQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.17.0.3' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@2ac0f5dd31bf:~# exit
logout
Connection to 172.17.0.3 closed.
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contrarization And Devops_Lab/Lab/Lab-9$
```


Step 8: Create Ansible Inventory

Create the file `inventory.ini`.

```
nano inventory.ini
```

Add the following configuration:

```
[servers]
172.17.0.3
172.17.0.4
172.17.0.5
172.17.0.6

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

Step 9: Test Ansible Connectivity

Ping all servers using Ansible.

```
ansible all -i inventory.ini -m ping
```

```
divyanshubuntu@DESKTOP-GVHDQHP: /mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-9$ export ANSIBLE_HOST_KEY_CHECKING=False
divyanshubuntu@DESKTOP-GVHDQHP: /mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-9$ ansible all -i inventory.ini -m ping
172.17.0.3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.5 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.4 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Step 10: Create Ansible Playbook

Create a playbook file.

```
nano playbook1.yml
```

Add the following content:

```
---
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install required packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on "
```

Step 11: Run the Playbook

Execute the playbook to configure all servers.

```
ansible-playbook -i inventory.ini playbook1.yml
```

This will:

- Update package lists
- Install required packages
- Create a test file on each server

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contratization And Devops_Lab/Lab/Lab-9$ ansible-playbook -i inventory.ini playbook1.yml

PLAY [Configure multiple servers] *****

TASK [Gathering Facts] *****
ok: [172.17.0.5]
ok: [172.17.0.3]
ok: [172.17.0.4]
ok: [172.17.0.2]

TASK [Update apt packages] *****
ok: [172.17.0.4]
ok: [172.17.0.3]
ok: [172.17.0.2]
ok: [172.17.0.5]

TASK [Install packages] *****
changed: [172.17.0.3]
changed: [172.17.0.4]
changed: [172.17.0.5]
changed: [172.17.0.2]

TASK [Create test file] *****
changed: [172.17.0.3]
changed: [172.17.0.5]
changed: [172.17.0.2]
changed: [172.17.0.4]

PLAY RECAP *****
172.17.0.2      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.3      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.4      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.5      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Step 12: Verify Configuration

Check the created file using Ansible.

```
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```

Or verify using Docker:

```
for i in {1..4}; do
docker exec server${i} cat /root/ansible_test.txt
done
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-9$ ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
172.17.0.2 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.2
172.17.0.3 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.3
172.17.0.4 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.4
172.17.0.5 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.5
```

Step 13: Cleanup

Remove all containers after completing the experiment.

```
for i in {1..4}; do
docker rm -f server${i}
done
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-9$ for i in {1..4}; do
docker rm -f server${i}
done
server1
server2
server3
server4
```

Experiment 10 — SonarQube Code Quality Analysis

Overview

SonarQube is an open-source platform for **continuous inspection of code quality**. It performs automatic reviews using static analysis to detect:

- Bugs
- Code Smells
- Security Vulnerabilities

Why SonarQube?

Feature	Description
Continuous Inspection	Scans code with every commit, providing immediate feedback
Quality Gates	Defines pass/fail criteria for code quality
Technical Debt Quantification	Measures effort needed to fix issues
Multi-language Support	Supports 20+ programming languages
Visual Analytics	Dashboard showing code quality metrics and trends

Hands-On Lab

Step 1 — Create `docker-compose.yml`

```
nano docker-compose.yml
```

```
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

networks:
  sonarqube-lab:
    driver: bridge
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-10$ cat docker-compose.yml
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR JDBC URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR JDBC USERNAME: sonar
      SONAR JDBC PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:
```

Step 2 — Start the Containers

```
docker-compose up -d
```

```
lts-community: Pulling from library/sonarqube
60d98d907669: Pull complete
e24a8b9e652f: Pull complete
f3929ce9ef98: Pull complete
1df735f481ad: Pull complete
5d5a1fad7028: Pull complete
eb27e3a98da1: Pull complete
c7ad1fe61e07: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
Creating sonar-db ... done
Creating sonarqube ... done
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-10$
```

Step 3 — Verify SonarQube via Logs

```
docker-compose logs -f sonarqube
```

```
divyanshubuntu@DESKTOP-GVHQDHP:/mnt/f/Contraintization And DevOps/Contraintization And DevOps_Lab/Lab/Lab-10$ docker-compose logs -f sonarqube
Attaching to sonarqube
sonarqube | 2026.04.24 16:59:24 INFO app[[o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
sonarqube | 2026.04.24 16:59:24 INFO app[[o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9000, TCP: 127.0.0.1:36101]
sonarqube | 2026.04.24 16:59:24 INFO app[[o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]: /opt/sonarqube/elasticsearch/bin/elasticsearch
sonarqube | 2026.04.24 16:59:24 INFO app[[o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
sonarqube | 2026.04.24 16:59:26 INFO es[[o.e.n.Node] version[7.17.15], pid[26], build[default/tar/0b8ecfb4378335f4689c4223d1f1115f16bef3ba/2023-11-10T22:03:46.987399016Z], OS[linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8]
sonarqube | 2026.04.24 16:59:26 INFO es[[o.e.n.Node] JVM home [/opt/java/openjdk]
sonarqube | 2026.04.24 16:59:26 INFO es[[o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/sonarqube/logs/es_hs_err_pid%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.headless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimization=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog4j.shutdownHookEnabled=false, -Dlog4j2.disable.jmx=true, -Dlog4j2.formatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -Des.enforce.bootstrap.checks=true, -Xms512m, -Xmx512m, -XX:MaxDirectMemorySize=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqube/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distribution.flavor=default, -Des.distribution.type=tar, -Des.bundled.jdk=false]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.p.PluginsService] loaded module [analysis-common]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.p.PluginsService] loaded module [lang-painless]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.p.PluginsService] loaded module [parent-join]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.p.PluginsService] loaded module [reindex]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.p.PluginsService] loaded module [transport-netty4]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.p.PluginsService] no plugins loaded
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.e.NodeEnvironment] using [1] data paths, mounts [[/opt/sonarqube/data (/dev/sdd)], net usable_space [937.2gb], net total_space [1006.8gb], types [ext4]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.e.NodeEnvironment] heap size [512mb], compressed ordinary object pointers [true]
sonarqube | 2026.04.24 16:59:27 INFO es[[o.e.n.Node] node name [sonarqube], node ID [P-fLU-KQRpQW3LxChL12A], cluster name [sonarqube], roles [data_frozen, master, remote_cluster_client, data, data_content, data_hot, data_warm, data_cold, ingest]
sonarqube | 2026.04.24 16:59:28 WARN es[[o.e.t.ThreadPool] absolute clock went backwards by [1.4s/1459ms] while timer thread was sleeping
sonarqube | 2026.04.24 16:59:29 INFO es[[o.e.t.NettyAllocator] creating NettyAllocator with the following configs: [name=unpooled, suggested_max_allocation_size=256kb, factors={es.unsafe.use_unpooled_allocator=null, g1gc_enabled=true, g1gc_region_size=1mb, heap_size=512mb}]
sonarqube | 2026.04.24 16:59:29 INFO es[[o.e.i.r.RecoverySettings] using rate limit [40mb] with [default=40mb, read=0b, write=0b, max=0b]
sonarqube | 2026.04.24 16:59:29 INFO es[[o.e.d.DiscoveryModule] using discovery type [zen] and seed hosts providers [settings]
sonarqube | 2026.04.24 16:59:30 INFO es[[o.e.g.DanglingIndicesState] gateway.auto_import_dangling_indices is disabled, dangling indices will not be automatically detected or imported and must be managed manually
sonarqube | 2026.04.24 16:59:30 INFO es[[o.e.n.Node] initialized
sonarqube | 2026.04.24 16:59:30 INFO es[[o.e.n.Node] starting
```

Step 4 — Login to SonarQube

Open in browser:

```
http://localhost:9000
```


Field	Value
Username	admin
Password	admin

Log in to SonarQube

[Log In](#)
[Cancel](#)

Step 5 — SonarQube Home Page

localhost:9000/projects/create

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately.

[Learn More](#)

sonarqube

Projects Issues Rules Quality Profiles Quality Gates Administration

Search for projects...

How do you want to create your project?

Do you want to benefit from all of SonarQube's features (like repository import and Pull Request decoration)? Create your project from your favorite DevOps platform. First, you need to set up a DevOps platform configuration.

From Azure DevOps

Set up global configuration

From Bitbucket Server

Set up global configuration

From Bitbucket Cloud

Set up global configuration

From GitHub

Set up global configuration

From GitLab

Set up global configuration

Are you just testing or have an advanced use-case? Create a project manually

Manually

Get the most out of SonarQube!

+

Take advantage of the whole ecosystem by using SonarLint, a free IDE plugin that helps you find and fix issues earlier in your workflow. Connect SonarLint to SonarQube to sync rule sets and issue states.

[Learn More](#)

Dismiss

Step 6 — Create a Sample Java App

```
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

```
PS F:\Contratization and DevOps\Contraization And Devops_Lab\Lab-10> wsl
divyanshu@ubuntu:~/mnt/f/Contratization and DevOps/Contraization And Devops_Lab/Lab-10$ mkdir -p sample-java-app/src/main/java/com/example
divyanshu@ubuntu:~/mnt/f/Contratization and DevOps/Contraization And Devops_Lab/Lab-10$ cd sample-java-app
divyanshu@ubuntu:~/mnt/f/Contratization and DevOps/Contraization And Devops_Lab/Lab-10/sample-java-app$
```

Step 7 — Create `Calculator.java`

```
nano src/main/java/com/example/Calculator.java
```

```
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public String getName(String name) {
        return name.toUpperCase();
    }
}
```

```

    public String getName(String name) {
        return name.toUpperCase();
    }

    public void riskyOperation() {
        try {
            int x = 10 / 0;
        }
    }
}

```

Note: This file is intentionally crafted with common issues so SonarQube can detect and report them.

```

divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOps/Contraization And Devops_Lab/Lab/Lab-10/sample-java-app$ cat src/main/java/com/example/Calculator.java
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }
}

```

Step 8 — Create pom.xml

```
nano pom.xml
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN</sonar.login>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>
```

```
divyanshu@buntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab-10/sample-java-app$ cat pom.xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN</sonar.login>
  </properties>

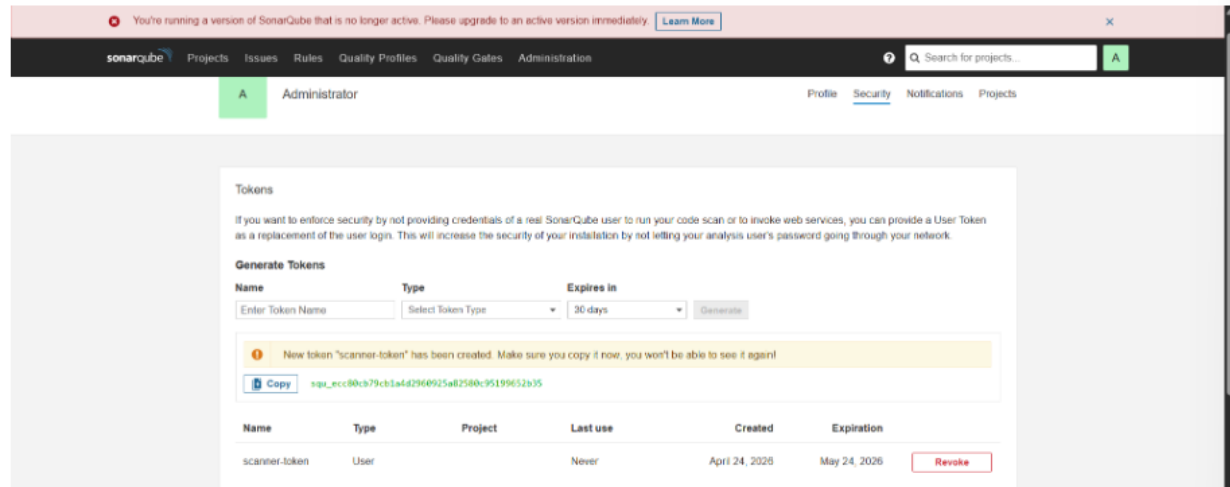
  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Step 9 — Generate a Scanner Token

Navigate to:

Profile → My Account → Security → Generate Token

- **Token Name:** scanner-token
- Click **Generate** and copy the token



Step 10 — Run the Sonar Scanner

```
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

Replace `YOUR_TOKEN` with the token generated in Step 9.

```
divyanshubuntu@DESKTOP-GVHDQ9P:/mnt/f/Contrarization and DevOPS/Contraization And Devops_Lab/Lab-10/sample-java-app$ export SONAR_TOKEN=squ_ecc80cb79cb1a4d2960925a82580c95199652b35

mvn sonar:sonar \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.login=$SONAR_TOKEN
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- sonar-maven-plugin:3.9.1.2184:sonar (default-cli) @ sample-app ---
[INFO] User cache: /home/divyanshubuntu/.sonar/cache
[INFO] SonarQube version: 9.9.8.100196
[INFO] Default locale: "en", source code encoding: "UTF-8" (analysis is platform dependent)
[INFO] Load global settings
[INFO] Load global settings (done) | time=165ms
[INFO] Server id: 74C15348-AZ3Abtb-IUCNJACU9UXD
[INFO] User cache: /home/divyanshubuntu/.sonar/cache
[INFO] Load/download plugins
[INFO] Load plugins index
[INFO] Load plugins index (done) | time=205ms
[INFO] Load/download plugins (done) | time=3043ms
[INFO] Process project properties
[INFO] Process project properties (done) | time=17ms
[INFO] Execute project builders
[INFO] Execute project builders (done) | time=1ms
[INFO] Project key: sample-java-app
[INFO] Base dir: /mnt/f/Contrarization and DevOPS/Contraization And Devops_Lab/Lab-10/sample-java-app
[INFO] Working dir: /mnt/f/Contrarization and DevOPS/Contraization And Devops_Lab/Lab-10/sample-java-app/target/sonar
[INFO] Load project settings for component key: 'sample-java-app'
[INFO] Load quality profiles
[INFO] Load quality profiles (done) | time=75ms
[INFO] Load active rules
[INFO] Load active rules (done) | time=2355ms
[INFO] Load analysis cache
[INFO] Load analysis cache (404) | time=13ms
[INFO] Load project repositories
[INFO] Load project repositories (done) | time=11ms
[INFO] Indexing files...
```

Step 11 — Confirm Build Success

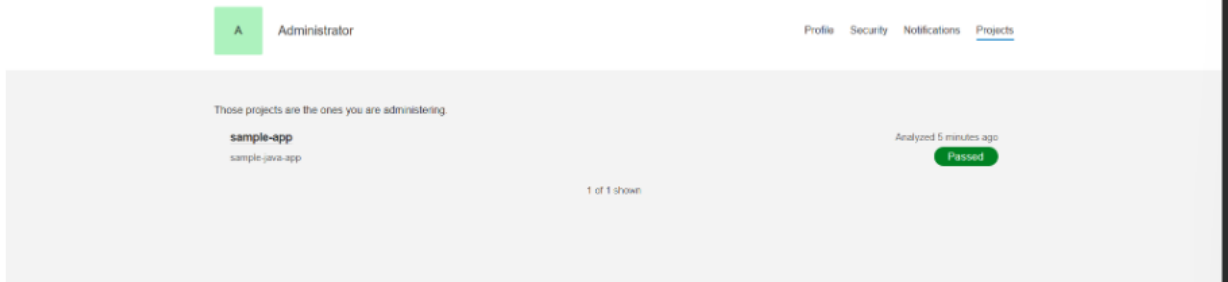
Scroll down in the terminal output to verify:

```
[INFO] BUILD SUCCESS
```

```
[INFO] 0 source files to be analyzed
[INFO] 0/0 source files have been analyzed
[INFO] Sensor IaC Docker Sensor [iac] (done) | time=71ms
[INFO] ----- Run sensors on project
[INFO] Sensor Analysis Warnings import [csharp]
[INFO] Sensor Analysis Warnings import [csharp] (done) | time=1ms
[INFO] Sensor Zero Coverage Sensor
[INFO] Sensor Zero Coverage Sensor (done) | time=19ms
[INFO] Sensor Java CPD Block Indexer
[INFO] Sensor Java CPD Block Indexer (done) | time=28ms
[INFO] SCM Publisher SCM provider for this project is: git
[INFO] SCM Publisher 2 source files to be analyzed
[WARNING] Thread[#50,JGit-FileStoreAttributeReader-1,5,main]: got smaller file timestamp on /mnt/f (F:\), /mnt/f/Contratization and DevOPS/Contratization And DevOps_Lab/Lab/.git: 2026-04-24T17:25:58Z < 2026-04-24T17:25:58.6057209Z. Aborting measurement at resolution PT0.3942791S.
[INFO] SCM Publisher 0/2 source files have been analyzed (done) | time=460ms
[WARNING] Missing blame information for the following files:
[WARNING] * src/main/java/com/example/Calculator.java
[WARNING] * pom.xml
[WARNING] This may lead to missing/broken features in SonarQube
[INFO] CPD Executor Calculating CPD for 1 file
[INFO] CPD Executor CPD calculation finished (done) | time=10ms
[INFO] Analysis report generated in 251ms, dir size=128.3 kB
[INFO] Analysis report compressed in 940ms, zip size=19.3 kB
[INFO] Analysis report uploaded in 618ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=sample-java-app
[INFO] Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] More about the report processing at http://localhost:9000/api/ce/task?id=AZ3AhunaTUCNjACU94Qe
[INFO] Analysis total time: 11.037 s
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 15.665 s
[INFO] Finished at: 2026-04-24T17:26:00Z
[INFO] -----
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contratization And DevOps_Lab/Lab-10/sample-java-app$
```

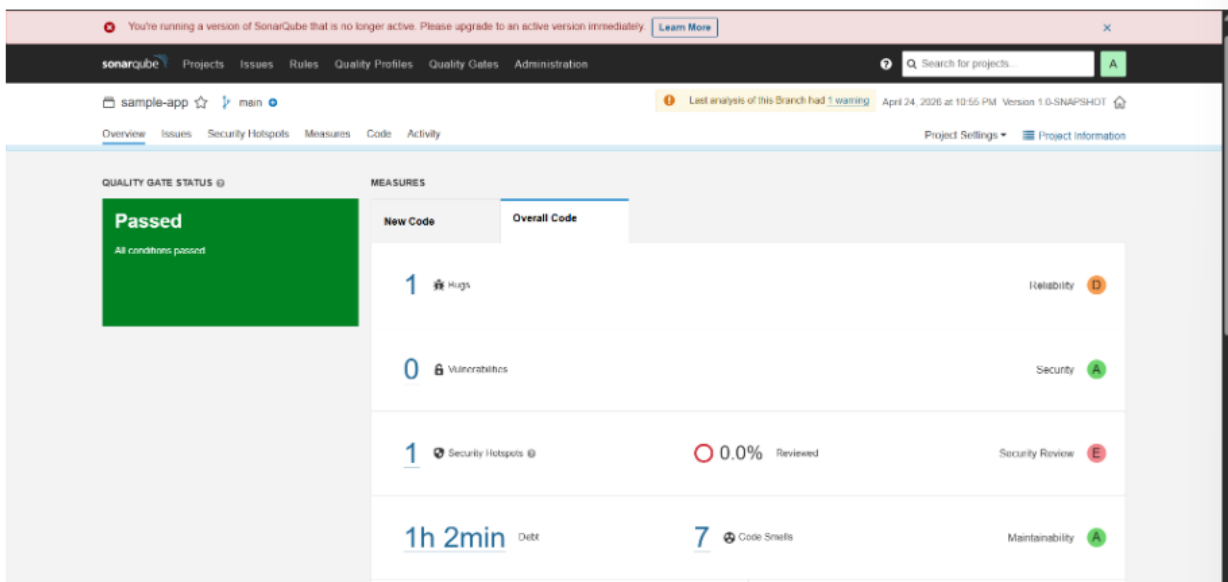
Step 12 — View Results in Browser

Open SonarQube dashboard at `http://localhost:9000` — your project analysis will be listed.



Step 13 — Explore the Full Report

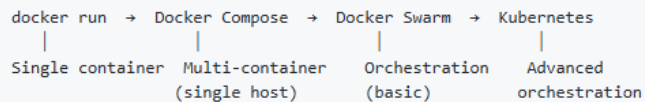
Click on your project to drill into the detailed quality report including bugs, vulnerabilities, code smells, and duplications.



Experiment 11: Orchestration via Docker Swarm

Introduction

The Progression Path



This experiment focuses on: Moving from Compose → Swarm

Step-1:- Create a `docker-compose.yml`

```
nano docker-compose.yml
```

Paste This:

```
version: '3.9'

services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

```
divyanshubunt@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-11$ cat docker-compose.yml
# docker-compose.yml (from Experiment 6)
version: '3.9'

services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html
```

Step-2:- Initialize Swarm

```
docker swarm init
```

```
divyanshubunt@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-11$ docker swarm init --advertise-addr 172.28.161.94
Swarm initialized: current node (dkjxqb6lp09qjswzld24z0npf) is now a manager.
```

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-29za5mgfcn0c8pc02au3xup83tn4t3ji9uj00qho26sl5pb38-akuzcyh22gf7b9dpx07cqdl7j 172.28.161.94:2377
```

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

```
divyanshubunt@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-11$
```

Step-3:- Verify Swarm is Active

```
docker node ls
```

```
divyanshubunt@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-11$ docker node ls
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
dkjxqb6lp09qjswzld24z0npf *  DESKTOP-GVHDQHP  Ready    Active           Leader             20.7.2
divyanshubunt@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-11$
```

Step-4:- Deploy as a stack

```
docker stack deploy -c docker-compose.yml wpstack
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$
```

Step-5:- Verify by listing Services

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$ docker service ls
ID            NAME              MODE                REPLICAS            IMAGE              PORTS
eek99lcm8m6   wpstack_db        replicated          1/1                 mysql:5.7
0fv6k4jx5g28  wpstack_wordpress replicated          1/1                 wordpress:latest   *:8080->80/tcp
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$
```

Step-6:- Details for container service

```
docker service ps wpstack_wordpress
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$ docker service ps wpstack_wordpress
ID            NAME              IMAGE              NODE                DESIRED STATE       CURRENT STATE          ERROR             PORTS
nwslvqrcxgn9  wpstack_wordpress.1  wordpress:latest  DESKTOP-GVHDQHP    Running             Running about a minute ago
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$
```

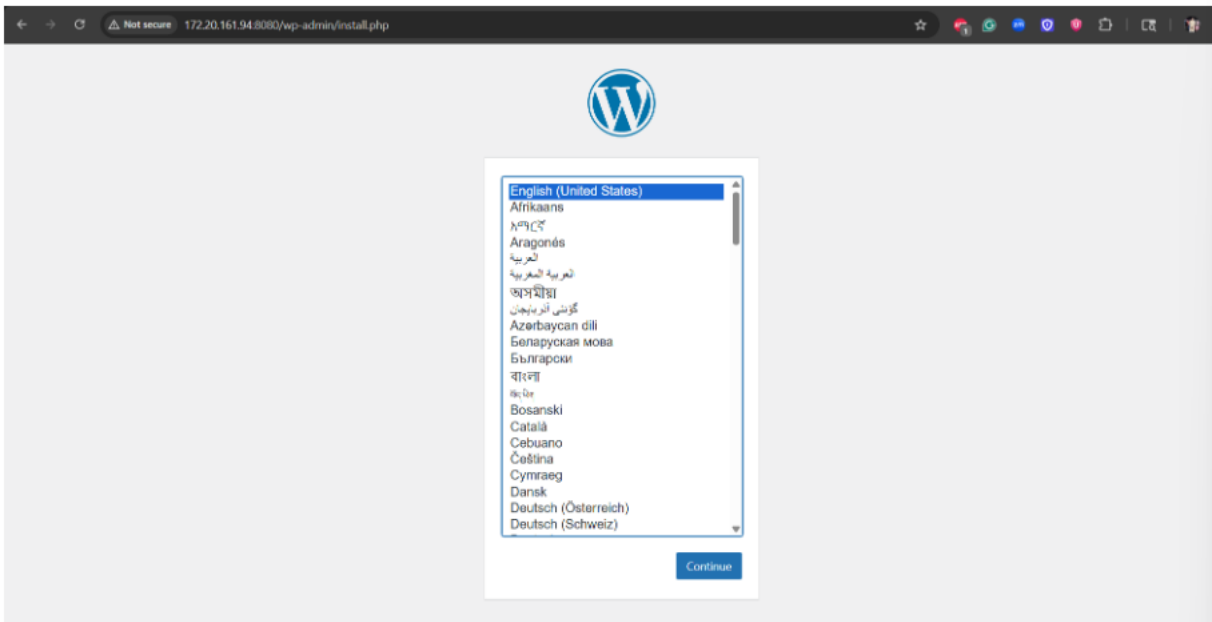
Step-7:- List Containers

```
docker ps
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
AMES          484ac62a844c   wordpress:latest        "docker-entrypoint.s..." About a minute ago Up About a minute 80/tcp
pstack_wordpress.1.nwslvqrcxgn99v64ooawo9heq   af986ec7b274   mysql:5.7              "docker-entrypoint.s..." About a minute ago Up About a minute 3306/tcp, 33060/tcp
pstack_db.1.gpdpqps9xhgfr5msfb9q12bvfb1a77bec2779  jenkins/jenkins:lts  "/usr/bin/tini -- /u..." 2 weeks ago    Up 8 minutes
enkins        f6e3d9adc07a   assignment_1-backend    "docker-entrypoint.s..." 4 weeks ago    Up 8 minutes
ode_backend   785a12392bf4   assignment_1-database    "docker-entrypoint.s..." 4 weeks ago    Up 8 minutes (healthy)
ostgres_db    1ccc305816d9   ghcr.io/k3d-io/k3d-proxy:5.8.3  "/bin/sh -c nginx-pr..." 6 weeks ago    Up 10 seconds  80/tcp, 0.0.0.0:41983->6443/tcp
3d-mycluster-serverlb  6e8bf745da79   advancedtask-fastapi-app  "uvicorn main:app --..." 8 weeks ago    Up 8 minutes  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
astapi-app
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$
```

Step-8:- Verify application on Containers

http://172.20.161.94:8080



Step-9:- Scale application Containers

```
docker service scale wpstack_wordpress=3
```

```
divyanshubuntu@DESKTOP-GVNDQHP: /mnt/4/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-11$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
divyanshubuntu@DESKTOP-GVNDQHP: /mnt/4/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-11$
```

Step-10:- Verify Scalling

```
docker service ls
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contraization And Devops_Lab/Lab/Lab-11$ docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
e4nvtal9cuwp      wpstack_db           replicated          1/1                 mysql:5.7
rfobhyy5xu9       wpstack_wordpress    replicated          3/3                 wordpress:latest    *:8080->80/tcp
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contraization And Devops_Lab/Lab/Lab-11$
```

Step-11:- List Wordpress Containers

```
docker ps | grep wordpress
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contraization And Devops_Lab/Lab/Lab-11$ docker ps | grep wordpress
e75c7f1774fd      wordpress:latest    "docker-entrypoint.s..." About a minute ago   Up 59 seconds        80/tcp
wpstack_wordpress.3.xcxmzy7hk4mohhmewzgx2im0r
263ef2c7e660      wordpress:latest    "docker-entrypoint.s..." About a minute ago   Up 59 seconds        80/tcp
wpstack_wordpress.2.kx24rlvbgkintit62p4fiksgp
973c573dbe0b      wordpress:latest    "docker-entrypoint.s..." 2 minutes ago       Up 2 minutes         80/tcp
wpstack_wordpress.1.twb963z9pe4gonvkjwgs48jsb
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contraization And Devops_Lab/Lab/Lab-11$
```

Step-12:- Kill any Container

```
docker kill <container-id>
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contraization And Devops_Lab/Lab/Lab-11$ docker kill e75c7f1774fd
e75c7f1774fd
```

Step-13:- Watch Swarm Recreating Contrainer

```
docker service ps wpstack_wordpress
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contratization and DevOPS/Contraization And Devops_Lab/Lab/Lab-11$ docker service ps wpstack_wordpress
ID                NAME                IMAGE                NODE                DESIRED STATE       CURRENT STATE       ERROR                PORTS
twb963z9pe4g      wpstack_wordpress.1  wordpress:latest     DESKTOP-GVHDQHP     Running              Running 30 minutes ago
lx24rlvbgkln      wpstack_wordpress.2  wordpress:latest     DESKTOP-GVHDQHP     Running              Running 29 minutes ago
1pbg57j9j1oi      wpstack_wordpress.3  wordpress:latest     DESKTOP-GVHDQHP     Running              Running 3 seconds ago
xcxmzy7hk4mo      \ wpstack_wordpress.3  wordpress:latest     DESKTOP-GVHDQHP     Shutdown             Failed 6 seconds ago  "task: non-zero exit (137)"
```

Step-14:- Verify Container running

```
docker ps | grep wordpress
```

```
divyanshu@buntu@DESKTOP-GVMDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$ docker ps | grep wordpress
2837576ec9ef    wordpress:latest    "docker-entrypoint.s..." 28 seconds ago    Up 25 seconds      80/tcp            wpst
ack_wordpress.3.1pbgs7j9j1oiojeilwt9eusk7
263ef2c7e660    wordpress:latest    "docker-entrypoint.s..." 29 minutes ago    Up 29 minutes      80/tcp            wpst
ack_wordpress.2.kx24rlvbgkintit62p4fiksgp
973c573db0eb    wordpress:latest    "docker-entrypoint.s..." 30 minutes ago    Up 30 minutes      80/tcp            wpst
ack_wordpress.1.1et696379ue4uorvk4mrs48ish
```

Step-15:- Remove Stack

```
docker stack rm wpstack
```

```
divyanshu@buntu@DESKTOP-GVMDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-11$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Conclusion

Summary

You started with	You can now do
Single container (<code>docker run</code>)	Multi-container (Compose)
Manual scaling	One-command scaling (<code>scale</code>)
Manual recovery	Automatic self-healing
Single host	Multi-host cluster ready

Final Takeaway

Compose defines the application. Swarm runs it reliably.

Experiment 12: Kubernetes

Introduction

Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

Core Kubernetes Concepts (Simple Explanation)

Docker Concept	Kubernetes Equivalent	What it means
Container	Pod	A pod is a group of one or more containers. Smallest unit in K8s.
Compose service	Deployment	Describes how your app should run (e.g., 2 copies, which image to use)
Load balancing	Service	Exposes your app to the outside world or other pods
Scaling	ReplicaSet	Ensures a certain number of pod copies are always running

Step-1:- Create a `wordpress-deployment.yaml`

```
nano wordpress-deployment.yaml
```

Paste This:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-12$ cat wordpress-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-12$
```

Step-2:- Apply Deployment

```
kubectl apply -f wordpress-deployment.yaml
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-12$ kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOPS/Containerization And Devops_Lab/Lab/Lab-12$
```


Step-3:- Create `wordpress-service.yaml`

```
nano wordpress-service.yaml
```

Paste this:-

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

```
PS F:\Contarization and DevOPS\Contraization And DEvops_Lab\Lab\Lab-12> wsl
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab\Lab\Lab-12$ cat wordpress-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab\Lab\Lab-12$
```

Step-4:- Deploy Service

```
kubectl apply -f wordpress-service.yaml
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab\Lab\Lab-12$ kubectl apply -f wordpress-service.yaml
service/wordpress-service created
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Contarization and DevOPS/Contraization And DEvops_Lab\Lab\Lab-12$
```

Step-5:- Get pods

```
kubectl get pods
```

```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-12$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web                                  1/1     Running   8 (3m29s ago)    30d
web-54466ccbf7-2dzs9                1/1     Running   8 (3m29s ago)    30d
web-54466ccbf7-b85wd                1/1     Running   8 (3m29s ago)    30d
web-54466ccbf7-p7988                1/1     Running   8 (3m29s ago)    30d
wordpress-7d6fedb8d8-nvldn          1/1     Running   0           2m27s
wordpress-7d6fedb8d8-v8gwp          1/1     Running   0           2m27s
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-12$
```

Step-6:- Check service

```
kubectl get svc
```

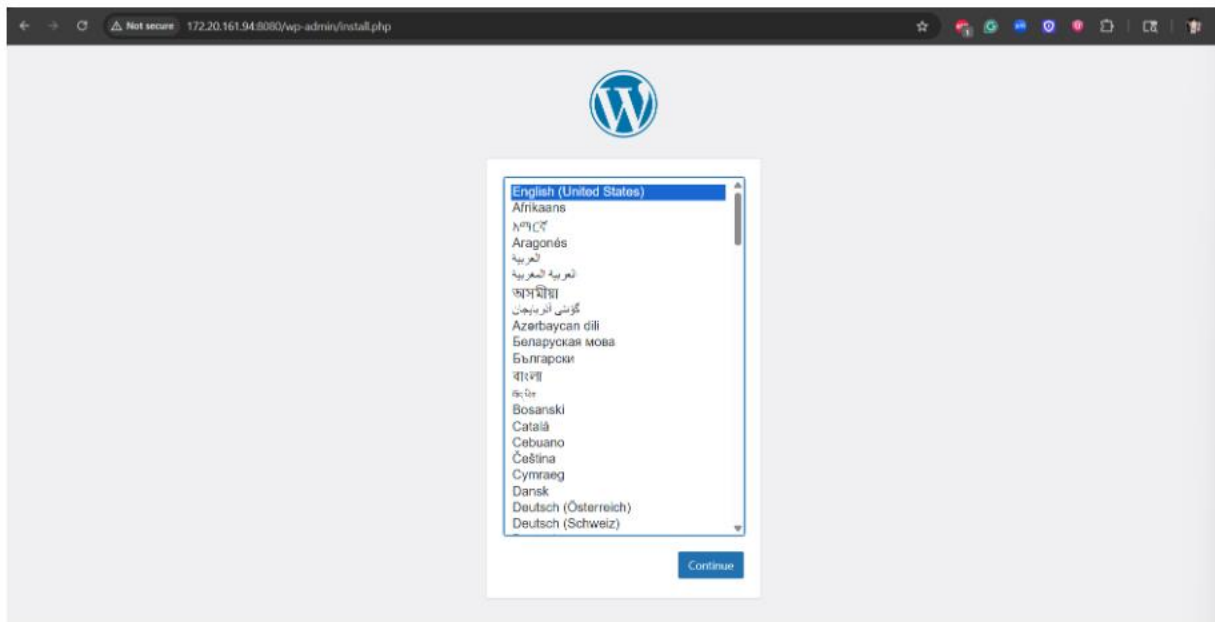
```
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-12$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.43.0.1    <none>         443/TCP           43d
web                  NodePort    10.43.48.239 <none>         80:32636/TCP     42d
wordpress-service    NodePort    10.43.24.254 <none>         80:30007/TCP     39s
divyanshuubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And Devops_Lab/Lab/Lab-12$
```

Step-7:- Verify On Browser

```
http://172.20.161.94:8080
```

Step-7:- Verify On Browser

http://172.20.161.94:8080



Step-8:- Scale Deployment

```
kubectl scale deployment wordpress --replicas=4
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-12$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Containerization and DevOps/Containerization And DevOps_Lab/Lab/Lab-12$
```

Step-9:- Get Pods

```
kubectl get pods
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraization And Devops_Lab/Lab/Lab-12$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web                                  1/1     Running   8 (5m6s ago)   30d
web-54466ccbf7-2dzsq                1/1     Running   8 (5m6s ago)   30d
web-54466ccbf7-b85wd                1/1     Running   8 (5m6s ago)   30d
web-54466ccbf7-p7988                1/1     Running   8 (5m6s ago)   30d
wordpress-7d6f6db8d8-5lptb          1/1     Running   0           21s
wordpress-7d6f6db8d8-nvldn          1/1     Running   0           4m4s
wordpress-7d6f6db8d8-v8gmp          1/1     Running   0           4m4s
wordpress-7d6f6db8d8-whw89          1/1     Running   0           21s
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraization And Devops_Lab/Lab/Lab-12$
```

Step-10:- Delete Pods to check Self-healing

```
kubectl delete pod <pod-name>
kubectl get pods
```

```
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraization And Devops_Lab/Lab/Lab-12$ kubectl delete pod wordpress-7d6f6db8d8-whw89
pod "wordpress-7d6f6db8d8-whw89" deleted from default namespace
divyanshubuntu@DESKTOP-GVHDQHP:/mnt/f/Contrarization and DevOPS/Contraization And Devops_Lab/Lab/Lab-12$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web                                  1/1     Running   8 (16m ago)   30d
web-54466ccbf7-2dzsq                1/1     Running   8 (16m ago)   30d
web-54466ccbf7-b85wd                1/1     Running   8 (16m ago)   30d
web-54466ccbf7-p7988                1/1     Running   8 (16m ago)   30d
wordpress-7d6f6db8d8-5j9bz          1/1     Running   0           4s
wordpress-7d6f6db8d8-nvldn          1/1     Running   0           15m
wordpress-7d6f6db8d8-v8gmp          1/1     Running   0           15m
wordpress-7d6f6db8d8-xzvcj          1/1     Running   0           112s
```

Name- Divyanshu Singh

Sap-500122856

Batch-1 ccvt